Chapter 3 ▪ XML

The preface of the book *Essential XML* by Don Box et al. (Addison-Wesley, 2000) stated only half-jokingly: "The Extensible Markup Language (XML) has replaced Java, Design Patterns, and Object Technology as the software industry's solution to world hunger." This kind of hype is long gone but, as you will see in this chapter, XML is still a very useful technology for describing structured information. XML tools make it easy to process and transform that information. However, XML is not a silver bullet. You need domain-specific standards and code libraries to use it effectively. Moreover, far from making Java obsolete, XML works very well with Java. Since the late 1990s, IBM, Apache, and others have been instrumental in producing high-quality Java libraries for XML processing. Many of these libraries have now been integrated into the Java platform.

This chapter introduces XML and covers the XML features of the Java API. As always, we'll point out, along the way, when using XML is justified—and when you have to take it with a grain of salt and try solving your problems the old-fashioned way: through good design and code.

3.1. Introducing XML

In **Chapter 9 of Volume I**, you have seen the use of *property files* to describe the configuration of a program. A property file contains a set of name/value pairs, such as

```
fontname=Times Roman
fontsize=12
windowsize=400 200
color=0 50 100
```

You can use the `Properties` class to read in such a file with a single method call. That's a nice feature, but it doesn't really go far enough. In many cases, the information you want to describe has more structure than the property file format can comfortably handle. Consider the `fontname/fontsize` entries in the example. It would be more object-oriented to have a single entry:

```
font=Times Roman 12
```

But then, parsing the font description gets ugly as you have to figure out when the font name ends and the font size starts.

Property files have a single flat hierarchy. You can often see programmers work around that limitation with key names like

```
title.fontname=Helvetica
title.fontsize=36
body.fontname=Times Roman
body.fontsize=12
```

Another shortcoming of the property file format is the requirement that keys must be unique. To store a sequence of values, you need another workaround, such as

```
menu.item.1=Times Roman
menu.item.2=Helvetica
menu.item.3=Goudy Old Style
```

The XML format solves these problems. It can express hierarchical structures and is thus more flexible than the flat table structure of a property file.

An XML file for describing a program configuration might look like this:

```
<config>
   <entry id="title">
      <font>
         <name>Helvetica</name>
         <size>36</size>
      </font>
   </entry>
   <entry id="body">
      <font>
         <name>Times Roman</name>
         <size>12</size>
      </font>
   </entry>
   <entry id="background">
      <color>
         <red>0</red>
         <green>50</green>
         <blue>100</blue>
      </color>
   </entry>
</config>
```

The XML format allows you to express the hierarchy and record repeated elements without contortions.

The format of an XML file is straightforward. It looks similar to an HTML file. There is a good reason for that—both XML and HTML are descendants of the venerable Standard Generalized Markup Language (SGML).

SGML has been around since the 1970s for describing the structure of complex documents. It has been used with success in some industries that require ongoing maintenance of massive documentation—in particular, the aircraft industry. However, SGML is quite complex, so it has never caught on in a big way. Much of that complexity arises because SGML has two conflicting goals. SGML wants to make sure that documents are formed according to the rules for their document type, but it also wants to make data entry easy by allowing shortcuts that reduce typing. XML was designed as a simplified version of SGML for use on the Internet. As is often true, simpler is better, and XML has enjoyed the immediate and enthusiastic reception that has eluded SGML for so long.



Note: You can find a very nice version of the XML standard, with annotations by Tim Bray, at **https://www.xml.com/axml/axml.html**.

Even though XML and HTML have common roots, there are important differences between the two.

- Unlike HTML, XML is case-sensitive. For example, `<H1>` and `<h1>` are different XML tags.
- In HTML, you can omit end tags, such as `</p>` or `</li>`, if it is clear from the context where a paragraph or list item ends. In XML, you can never omit an end tag.
- In XML, elements that have a single tag without a matching end tag must end in a `/`, as in `<img src="coffeecup.png"/>`. That way, the parser knows not to look for a `</img>` tag.
- In XML, attribute values must be enclosed in quotation marks. In HTML, quotation marks are optional. For example, `<img src="my hamster.png" width=300>` is legal HTML but not legal XML. In XML, you have to use quotation marks: `width="300"`.
- In HTML, you can have attribute names without values, such as `<input type="radio" name="language" value="Java" checked>`. In XML, all attributes must have values, such as `checked="true"` or (ugh) `checked="checked"`.
- There are XML formulations for HTML versions 4 and 5 that are known as XHTML.

3.2. The Structure of an XML Document

An XML document should start with a header such as

```
<?xml version="1.0"?>
```

or

```
<?xml version="1.0" encoding="UTF-8"?>
```

Strictly speaking, a header is optional, but it is highly recommended.



Note: Since SGML was created for processing of real documents, XML files are called *documents* even though many of them describe data sets that one would not normally call documents.

The header can be followed by a *document type definition* (DTD), such as

```
<!DOCTYPE web-app PUBLIC
   "-//Sun Microsystems, Inc.//DTD Web Application 2.2//EN"
   "http://java.sun.com/j2ee/dtds/web-app_2_2.dtd">
```

DTDs are an important mechanism to ensure the correctness of a document, but they are not required. We will discuss them later in this chapter.

Finally, the body of the XML document contains the *root element*, which can contain other elements. For example,

```
<?xml version="1.0"?>
<!DOCTYPE config . . .>
<config>
   <entry id="title">
      <font>
         <name>Helvetica</name>
         <size>36</size>
      </font>
```

```
   </entry>
   . . .
</config>
```

An element can contain *child elements*, text, or both. In the preceding example, the `font` element has two child elements, `name` and `size`. The `name` element contains the text `"Helvetica"`.



Tip: It is best to structure your XML documents so that an element contains *either* child elements *or* text. Here's an example of what you should avoid:

```
<font>
   Helvetica
   <size>36</size>
</font>
```

This is called *mixed content* in the XML specification. As you will see later in this chapter, you can simplify parsing if you avoid mixed content.

XML elements can contain attributes, such as

```
<size unit="pt">36</size>
```

There is some disagreement among XML designers about when to use elements and when to use attributes. For example, it would seem easier to describe a font as

```
<font name="Helvetica" size="36"/>
```

compared to

```
<font>
   <name>Helvetica</name>
   <size>36</size>
</font>
```

However, attributes are much less flexible. Suppose you want to add units to the size value. If you use attributes, you will have to add the unit to the attribute value:

```
<font name="Helvetica" size="36 pt"/>
```

Ugh! Now you have to parse the string `"36 pt"`, just the kind of hassle that XML was designed to avoid. Adding an attribute to the `size` element is much cleaner:

```
<font>
   <name>Helvetica</name>
   <size unit="pt">36</size>
</font>
```

A commonly used rule of thumb is that attributes should be used only to modify the interpretation of a value, not to specify values. If you find yourself engaged in a metaphysical discussion about whether a particular setting is a modification of the interpretation of a value or not, just say "no" to attributes and use elements throughout. Many useful XML documents don't use attributes at all.



Note: In HTML, the rule for attribute usage is simple: If it isn't displayed on the web page, it's an attribute. For example, consider the hyperlink

```
<a href="http://java.sun.com">Java Technology</a>
```

The string `Java Technology` is displayed on the web page, but the URL of the link is not a part of the displayed page. However, the rule isn't all that helpful for most XML files because the data in an XML file aren't normally meant to be viewed by humans.

Elements and text are the "bread and butter" of XML documents. Here are a few other markup instructions that you might encounter:

- *Character references* have the form `&#decimalValue;` or `&#xhexValue;`. For example, the é character can be denoted with either of the following:

  ```
  &#233; &#xE9;
  ```

- *Entity references* have the form `&name;`. The entity references

  ```
  &lt; &gt; &amp; &quot; &apos;
  ```

  have predefined meanings: the less-than, greater-than, ampersand, quotation mark, and apostrophe characters. You can define other entity references in a DTD.
- *CDATA sections* are delimited by `<![CDATA[` and `]]>`. They are a special form of character data. You can use them to include strings that contain characters such as `< > &` without having them interpreted as markup, for example:

  ```
  <![CDATA[< & > are my favorite delimiters]]>
  ```

  CDATA sections cannot contain the string `]]>`. Use this feature with caution! It is too often used as a back door for smuggling legacy data into XML documents.
- *Processing instructions* are instructions for applications that process XML documents. They are delimited by `<?` and `?>`, for example

  ```
  <?xml-stylesheet href="mystyle.css" type="text/css"?>
  ```

  Every XML document starts with a processing instruction

  ```
  <?xml version="1.0"?>
  ```

- *Comments* are delimited by `<!--` and `-->`, for example

```
<!-- This is a comment. -->
```

Comments should not contain the string `--`. Comments should only be information for human readers. They should never contain hidden commands; use processing instructions for commands.

3.3. Parsing an XML Document

To process an XML document, you need to *parse* it. A parser is a program that reads a file, confirms that the file has the correct format, breaks it up into the constituent elements, and lets a programmer access those elements. The Java API supplies two kinds of XML parsers:

- Tree parsers, such as a Document Object Model (DOM) parser, that read an XML document into a tree structure.
- Streaming parsers, such as a Simple API for XML (SAX) parser, that generate events as they read an XML document.

DOM parsers are easier to use for most purposes, so we explain them first. You may consider a streaming parser if you process very long documents whose tree structures would use up a lot of memory, or if you are only interested in a few elements and don't care about their context. For more information, see **Section 3.7**.

The DOM parser interface is standardized by the World Wide Web Consortium (W3C). The `org.w3c.dom` package contains the definitions of interface types such as `Document` and `Element`. Different suppliers, such as the Apache Organization and IBM, have written DOM parsers whose classes implement these interfaces. The Java API for XML Processing (JAXP) actually makes it possible to plug in any of these parsers. But the JDK also comes with a DOM parser that is derived from the Apache parser.

To read an XML document, you need a `DocumentBuilder` object that you get from a `DocumentBuilderFactory` like this:

```
DocumentBuilderFactory factory = DocumentBuilderFactory.newInstance();
DocumentBuilder builder = factory.newDocumentBuilder();
```

You can now read a document from a file:

```
File f = . . .;
Document doc = builder.parse(f);
```

Alternatively, you can use a URL string:

```
String urlString = . . .;
Document doc = builder.parse(urlString);
```

You can even specify an arbitrary input stream:

```
InputStream in = . . .;
Document doc = builder.parse(in);
```



Note: If you use an input stream as an input source, the parser will not be able to locate other files that are referenced relative to the location of the document, such as a DTD in the same directory. You can install an "entity resolver" to overcome that problem. See **https://www.xml.com/pub/a/2004/03/03/catalogs.html** and **https://openjdk.org/jeps/268** for more information.

A `Document` object is an in-memory representation of the tree structure of an XML document. It is composed of objects whose classes implement the `Node` interface and its various subinterfaces. **Figure 3.1** shows the inheritance hierarchy of the subinterfaces.

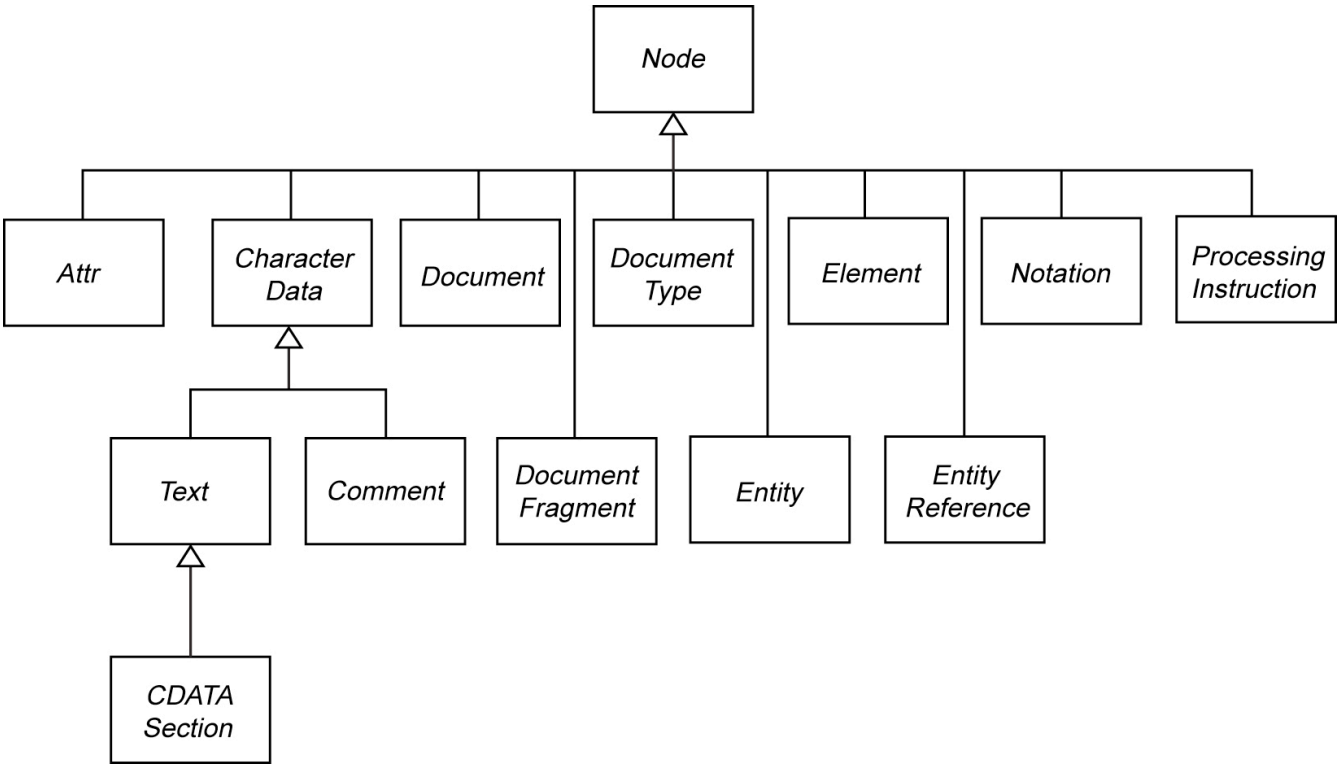


Figure 3.1: The `Node` interface and its subinterfaces

Start analyzing the contents of a document by calling the `getDocumentElement` method. It returns the root element.

```
Element root = doc.getDocumentElement();
```

For example, if you are processing a document

```
<?xml version="1.0"?>
<font>
   . . .
</font>
```

then calling `getDocumentElement` returns the `font` element.

The `getTagName` method returns the tag name of an element. In the preceding example, `root.getTagName()` returns the string `"font"`.

To get an element's children (which may be subelements, text, comments, or other nodes), use the `getChildNodes` method. That method returns a collection of type `NodeList`. That type was standardized before the standard Java collections, so it has a different access protocol. The `item` method gets the item with a

given index, and the `getLength` method gives the total count of the items. You can enumerate all children like this:

```
NodeList children = root.getChildNodes();
for (int i = 0; i < children.getLength(); i++) {
    Node child = children.item(i);
    . . .
}
```

Be careful when analyzing children. Suppose, for example, that you are processing the document

```
<font>
   <name>Helvetica</name>
   <size>36</size>
</font>
```

You would expect the `font` element to have two children, but the parser reports five:

- The whitespace between `<font>` and `<name>`
- The `name` element
- The whitespace between `</name>` and `<size>`
- The `size` element
- The whitespace between `</size>` and `</font>`

Figure 3.2 shows the DOM tree.

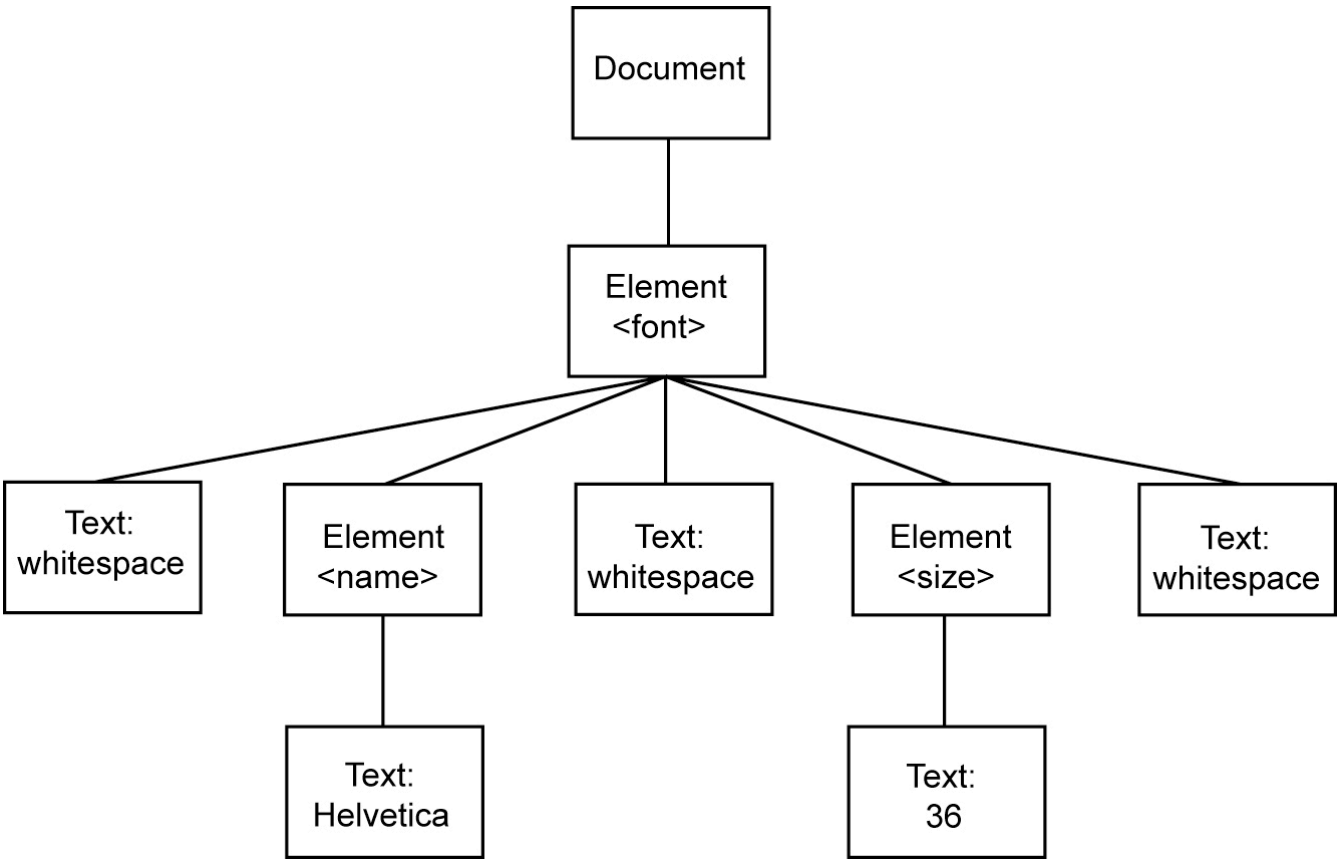


Figure 3.2: A simple DOM tree

If you expect only subelements, you can ignore the whitespace:

```
for (int i = 0; i < children.getLength(); i++) {
    Node child = children.item(i);
    if (child instanceof Element childElement) {
        . . .
    }
}
```

Now you look at only two elements, with tag names `name` and `size`.

As you will see in the next section, you can do even better if your document has a DTD. Then the parser knows which elements don't have text nodes as children, and it can suppress the whitespace for you.

When analyzing the `name` and `size` elements, you want to retrieve the text strings that they contain. Those text strings are themselves contained in child nodes of type `Text`. You know that these `Text` nodes are the only children, so you can use the `getFirstChild` method without having to traverse another `NodeList`. Then, use the `getData` method to retrieve the string stored in a `Text` node:

```
for (int i = 0; i < children.getLength(); i++) {
    Node child = children.item(i);
    if (child instanceof Element childElement) {
        var textNode = (Text) childElement.getFirstChild();
        String text = textNode.getData().strip();
        if (childElement.getTagName().equals("name"))
            name = text;
        else if (childElement.getTagName().equals("size"))
            size = Integer.parseInt(text);
    }
}
```



Tip: It is a good idea to call `strip` on the return value of the `getData` method. If the author of an XML file puts the beginning and the ending tags on separate lines, such as

```
 <size>
    36
 </size>
```

then the parser will include all line breaks and spaces in the text node data. Calling the `strip` method removes the whitespace surrounding the actual data.

You can also get the last child with the `getLastChild` method, and the next sibling of a node with `getNextSibling`. Therefore, another way of traversing a node's children is

```
for (Node childNode = element.getFirstChild();
        childNode != null;
        childNode = childNode.getNextSibling()) {
    . . .
}
```

To enumerate the attributes of a node, call the `getAttributes` method. It returns a `NamedNodeMap` object that contains `Node` objects describing the attributes. You can traverse the nodes in a `NamedNodeMap` the same way as

in a `NodeList`. Then, call the `getNodeName` and `getNodeValue` methods to get the attribute names and values.

```
NamedNodeMap attributes = element.getAttributes();
for (int i = 0; i < attributes.getLength(); i++) {
    Node attribute = attributes.item(i);
    String name = attribute.getNodeName();
    String value = attribute.getNodeValue();
    . . .
}
```

Alternatively, if you know the name of an attribute, you can retrieve the corresponding value directly:

```
String unit = element.getAttribute("unit");
```

You have now seen how to analyze a DOM tree. The program in **Listing 3.1** puts these techniques to work by converting an XML document to JSON format.

The tree display clearly shows how child elements are surrounded by text containing whitespace and comments. You can clearly see the newline and return characters as `\n`.

You don't have to be familiar with JSON to understand how the program works with the DOM tree. Simply observe the following:

- We use a `DocumentBuilder` to read a `Document` from a file.
- For each element, we print the tag name, attributes, and elements.
- For character data, we produce a string with the data. If the data comes from a comment, we add a `"Comment: "` prefix.

Listing 3.1 v2ch03/dom/JSONConverter.java

```
1  package v2ch03.dom;
2
3  import module java.base;
4  import module java.xml;
5  import org.w3c.dom.CharacterData;
6  import org.w3c.dom.Comment;
7
8  /**
9   * This program displays an XML document as a tree in JSON format.
10  */
11 class JSONConverter {
12     void main(String[] args) throws Exception {
13         String filename = args.length == 0 ? IO.readln("Input file: ") : args[0];
14         DocumentBuilderFactory factory = DocumentBuilderFactory.newInstance();
15         DocumentBuilder builder = factory.newDocumentBuilder();
16
17         Document doc = builder.parse(filename);
18         Element root = doc.getDocumentElement();
19         IO.println(convert(root, 0));
20     }
21
22     StringBuilder convert(Node node, int level) {
```

```
        if (node instanceof Element elem) {
            return elementObject(elem, level);
        }
        else if (node instanceof CharacterData cd) {
            return characterString(cd, level);
        }
        else {
            return pad(new StringBuilder(), level)
                    .append(jsonEscape(node.getClass().getName()));
        }
    }

    Map<Character, String> replacements = Map.of('\b', "\\b", '\f', "\\f", '\n',
            "\\n", '\r', "\\r", '\t', "\\t", '"', "\\\"", '\\', "\\\\");

    StringBuilder jsonEscape(String str) {
        var result = new StringBuilder("\"");
        for (int i = 0; i < str.length(); i++) {
            char ch = str.charAt(i);
            String replacement = replacements.get(ch);
            if (replacement == null)
                result.append(ch);
            else
                result.append(replacement);
        }
        result.append("\"");
        return result;
    }

    StringBuilder characterString(CharacterData node, int level) {
        var result = new StringBuilder();
        StringBuilder data = jsonEscape(node.getData());
        if (node instanceof Comment) data.insert(1, "Comment: ");
        pad(result, level).append(data);
        return result;
    }

    StringBuilder elementObject(Element elem, int level) {
        var result = new StringBuilder();
        pad(result, level).append("{\n");
        pad(result, level + 1).append("\"name\": ");
        result.append(jsonEscape(elem.getTagName()));
        NamedNodeMap attrs = elem.getAttributes();
        if (attrs.getLength() > 0) {
            pad(result.append(",\n"), level + 1).append("\"attributes\": ");
            result.append(attributeObject(attrs));
        }
        NodeList children = elem.getChildNodes();
        if (children.getLength() > 0) {
            pad(result.append(",\n"), level + 1).append("\"children\": [\n");
            for (int i = 0; i < children.getLength(); i++) {
                if (i > 0) result.append(",\n");
                result.append(convert(children.item(i), level + 2));
            }
            result.append("\n");
            pad(result, level + 1).append("]\n");
        }
        pad(result, level).append("}");
```

```
81          return result;
82      }
83
84      StringBuilder pad(StringBuilder builder, int level) {
85          for (int i = 0; i < level; i++)
86              builder.append("  ");
87          return builder;
88      }
89
90      StringBuilder attributeObject(NamedNodeMap attrs) {
91          var result = new StringBuilder("{");
92          for (int i = 0; i < attrs.getLength(); i++) {
93              if (i > 0) result.append(", ");
94              result.append(jsonEscape(attrs.item(i).getNodeName()));
95              result.append(": ");
96              result.append(jsonEscape(attrs.item(i).getNodeValue()));
97          }
98          result.append("}");
99          return result;
100     }
101 }
```

`javax.xml.parsers.DocumentBuilderFactory` **1.4**

- `static DocumentBuilderFactory newInstance()`
  returns an instance of the `DocumentBuilderFactory` class.
- `DocumentBuilder newDocumentBuilder()`
  returns an instance of the `DocumentBuilder` class.

`javax.xml.parsers.DocumentBuilder` **1.4**

- `Document parse(File f)`
- `Document parse(String url)`
- `Document parse(InputStream in)`
  parse an XML document from the given file, URL, or input stream and return the parsed document.

org.w3c.dom.Document **1.4**

- `Element getDocumentElement()`
  returns the root element of the document.

org.w3c.dom.Element **1.4**

- `String getTagName()`
  returns the name of the element.
- `String getAttribute(String name)`
  returns the value of the attribute with the given name, or an empty string if there is no such attribute.

org.w3c.dom.Node **1.4**

- `NodeList getChildNodes()`
  returns a node list that contains all children of this node.
- `Node getFirstChild()`
- `Node getLastChild()`
  get the first or last child node of this node, or `null` if this node has no children.
- `Node getNextSibling()`
- `Node getPreviousSibling()`
  get the next or previous sibling of this node, or `null` if this node has no siblings.
- `Node getParentNode()`
  gets the parent of this node, or `null` if this node is the document node.
- `NamedNodeMap getAttributes()`
  returns a node map that contains `Attr` nodes that describe all attributes of this node.
- `String getNodeName()`
  returns the name of this node. If the node is an `Attr` node, the name is the attribute name.
- `String getNodeValue()`
  returns the value of this node. If the node is an `Attr` node, the value is the attribute value.

org.w3c.dom.CharacterData **1.4**

- `String getData()`
  returns the text stored in this node.

org.w3c.dom.NodeList **1.4**

- `int getLength()`
  returns the number of nodes in this list.
- `Node item(int index)`
  returns the node with the given index. The index is between `0` and `getLength()` – 1.

org.w3c.dom.NamedNodeMap **1.4**

- `int getLength()`
  returns the number of nodes in this map.
- `Node item(int index)`
  returns the node with the given index. The index is between `0` and `getLength()` – 1.

3.4. Validating XML Documents

In the previous section, you saw how to traverse the tree structure of a DOM document. However, with that approach, you'll have to do quite a bit of tedious programming and error checking. It's not just having to deal with whitespace between elements; you will also need to check whether the document contains the nodes that you expect. For example, suppose you are reading this element:

```
<font>
   <name>Helvetica</name>
   <size>36</size>
</font>
```

You get the first child. Oops . . . it is a text node containing whitespace `"\n "`. You skip text nodes and find the first element node. Then, you need to check that its tag name is `"name"` and that it has one child node of type `Text`. You move on to the next nonwhitespace child and make the same check. What if the author of the document switched the order of the children or added another child element? It is tedious to code all this error checking—but reckless to skip the checks.

Fortunately, one of the major benefits of an XML parser is that it can automatically verify that a document has the correct structure. That makes parsing much simpler. For example, if you know that the `font` fragment has passed validation, you can simply get the two grandchildren, cast them as `Text` nodes, and get the text data, without any further checking.

To specify the document structure, you can supply a DTD or an XML Schema definition. A DTD or schema contains rules that explain how a document should be formed, by specifying the legal child elements and attributes for each element. For example, a DTD might contain a rule:

```
<!ELEMENT font (name,size)>
```

This rule expresses that a `font` element must always have two children, which are `name` and `size` elements. The XML Schema language expresses the same constraint as

```
<xsd:element name="font">
   <xsd:sequence>
      <xsd:element name="name" type="xsd:string"/>
      <xsd:element name="size" type="xsd:int"/>
   </xsd:sequence>
</xsd:element>
```

XML Schema can express more sophisticated validation conditions (such as the fact that the `size` element must contain an integer) than DTDs. Unlike the DTD syntax, the XML Schema syntax itself uses XML, which is a benefit if you need to process schema files.

In the next section, we will discuss DTDs in detail, then briefly cover the basics of XML Schema support. Finally, we will present a complete application that demonstrates how validation simplifies XML programming.

3.4.1. Document Type Definitions

There are several methods for supplying a DTD. You can include a DTD in an XML document like this:

```
<?xml version="1.0"?>
<!DOCTYPE config [
   <!ELEMENT config . . .>
   more rules
   . . .
]>
<config>
   . . .
</config>
```

As you can see, the rules are included inside a `DOCTYPE` declaration, in a block delimited by `[. . .]`. The document type must match the name of the root element, such as `config` in our example.

Supplying a DTD inside an XML document is somewhat uncommon because DTDs can grow lengthy. It makes more sense to store the DTD externally. The `SYSTEM` declaration can be used for that purpose. Specify a URL that contains the DTD, for example:

```
<!DOCTYPE config SYSTEM "config.dtd">
```

or

```
<!DOCTYPE config SYSTEM "http://myserver.com/config.dtd">
```



Caution: If you use a relative URL for the DTD (such as `"config.dtd"`), give the parser a `File` or `URL` object, not an `InputStream`. If you must parse from an input stream, supply an entity resolver (see the following note).

The mechanism for identifying well-known DTDs has its origin in SGML. Here is an example:

```
<!DOCTYPE web-app
   PUBLIC "-//Sun Microsystems, Inc.//DTD Web Application 2.2//EN"
   "http://java.sun.com/j2ee/dtds/web-app_2_2.dtd">
```

If an XML processor knows how to locate the DTD with the public identifier, it need not go to the URL.



Note: The system identifier URL of a DTD may not actually be working, or be purposefully slowed down. An example for the latter is the system identifier of the XHTML 1.0 Strict DTD, `http://www.w3.org/TR/xhtml1/DTD/xhtml1-strict.dtd`. If you parse an XHTML file, it may take a minute or two for the DTD to be served.

The remedy is to use an *entity resolver* that maps public identifiers to local files. You provide an object of a class that implements the `EntityResolver` interface and provides a `resolveEntity` method.

XML catalogs are a convenient mechanism for resolving entities. Provide one or more *catalog files* of the form

```
<?xml version="1.0"?>
<!DOCTYPE catalog PUBLIC "-//OASIS//DTD XML Catalogs V1.0//EN"
      "http://www.oasis-open.org/committees/entity/release/1.0/catalog.dtd">
<catalog xmlns="urn:oasis:names:tc:entity:xmlns:xml:catalog" prefer="public">
   <public publicId=". . ." uri=". . ."/>
   . . .
</catalog>
```

Then construct and install a resolver like this:

```
builder.setEntityResolver(CatalogManager.catalogResolver(
   CatalogFeatures.defaults(),
   Path.of("catalog.xml").toAbsolutePath().toUri()));
```

See **Listing 3.6** for a complete example.

> Instead of setting the catalog file locations in your program, you can specify them on the command line with the `javax.xml.catalog.files` system property. Provide absolute `file` URLs separated by semicolons.

Now that you have seen how the parser locates the DTD, let us consider the various kinds of rules.

The `ELEMENT` rule specifies what children an element can have. Use a regular expression, made up of the components shown in **#ch02table01**.

Table 3.1: Rules for Element Content

Rule	Meaning
$E*$	0 or more occurrences of E
$E+$	1 or more occurrences of E
$E?$	0 or 1 occurrences of E
$E_1 \| E_2 \| \ldots \| E_n$	One of E_1, E_2, . . ., E_n
E_1, E_2, . . ., E_n	E_1 followed by E_2, . . ., E_n
`#PCDATA`	Text
$(\#PCDATA \| E_1 \| E_2 \| \ldots \| E_n)*$	0 or more occurrences of text and E_1, E_2, . . ., E_n in any order (mixed content)
`ANY`	Any children allowed
`EMPTY`	No children allowed

Here are several simple but typical examples. The following rule states that a `menu` element contains 0 or more `item` elements:

```
<!ELEMENT menu (item)*>
```

This set of rules states that a font is described by a `name` followed by a `size`, each of which contain text:

```
<!ELEMENT font (name,size)>
<!ELEMENT name (#PCDATA)>
<!ELEMENT size (#PCDATA)>
```

The abbreviation `PCDATA` denotes *parsed character data*. It is "parsed" because the parser interprets the text string, looking for `<` characters that denote the start of a new tag, or `&` characters that denote the start of an entity.

An element specification can contain regular expressions that are nested and complex. For example, here is a rule that describes the makeup of a chapter in a book:

```
<!ELEMENT chapter (intro,(heading,(para|image|table|note)+)+)
```

Each chapter starts with an introduction, which is followed by one or more sections consisting of a heading and one or more paragraphs, images, tables, or notes.

However, in one common case you can't define the rules to be as flexible as you might like. Whenever an element can contain text, there are only two valid cases. Either the element contains nothing but text, such as

```
<!ELEMENT name (#PCDATA)>
```

or the element contains *any combination of text and tags in any order*, such as

```
<!ELEMENT para (#PCDATA|em|strong|code)*>
```

It is not legal to specify any other types of rules that contain `#PCDATA`. For example, the following is illegal:

```
<!ELEMENT captionedImage (image,#PCDATA)>
```

You have to rewrite such a rule, either by introducing another `caption` element or by allowing any combination of `image` elements and text.

This restriction simplifies the job of the XML parser when parsing *mixed content* (a mixture of tags and text). Since you lose some control by allowing mixed content, it is best to design DTDs so that all elements contain either other elements or nothing but text.



Note: Actually, it isn't quite true that you can specify arbitrary regular expressions of elements in a DTD rule. An XML parser may reject certain complex rule sets that lead to nondeterministic parsing. For example, a regular expression `((x,y)|(x,z))` is nondeterministic. When the parser sees `x`, it doesn't know which of the two alternatives to take. This expression can be rewritten in a deterministic form as `(x,(y|z))`. However, some expressions can't be reformulated, such as `((x,y)*|x?)`. The parser in the Java XML API gives no warnings when presented with an ambiguous DTD; it simply picks the first matching alternative when parsing, which causes it to reject some correct inputs. The parser is well within its rights to do so because the XML standard allows a parser to assume that the DTD is unambiguous.

You can also specify rules to describe the legal attributes of elements. The general syntax is

```
<!ATTLIST element attribute type default>
```

#ch02table02 shows the legal attribute types, and **#ch02table03** shows the syntax for the defaults.

Table 3.2: Attribute Types

Type	Meaning
`CDATA`	Any character string
$(A_1\|A_2\| \ldots \|A_n)$	One of the string attributes A_1, A_2, . . ., A_n

Type	Meaning
`NMTOKEN`, `NMTOKENS`	One or more name tokens
`ID`	A unique ID
`IDREF`, `IDREFS`	One or more references to a unique ID
`ENTITY`, `ENTITIES`	One or more unparsed entities

Table 3.3: Attribute Defaults

Default	Meaning
`#REQUIRED`	Attribute is required.
`#IMPLIED`	Attribute is optional.
A	Attribute is optional; the parser reports it to be *A* if it is not specified.
`#FIXED` *A*	The attribute must either be unspecified or *A*; in either case, the parser reports it to be *A*.

Here are two typical attribute specifications:

```
<!ATTLIST font style (plain|bold|italic|bold-italic) "plain">
<!ATTLIST size unit CDATA #IMPLIED>
```

The first specification describes the `style` attribute of a `font` element. There are four legal attribute values, and the default value is `plain`. The second specification expresses that the `unit` attribute of the `size` element can contain any character data sequence.



Note: It is generally recommended to use elements, not attributes, to describe data. Thus, the font style should be a separate element, such as `<font><style>plain</style>. . .</font>`. However, attributes have an undeniable advantage for enumerated types because the parser can verify that the values are legal. For example, if the font style is an attribute, the parser checks that it is one of the four allowed values, and supplies a default if no value was given.

The handling of a `CDATA` attribute value is subtly different from the processing of `#PCDATA` that you have seen before, and quite unrelated to the `<![CDATA[. . .]]>` sections. The attribute value is first *normalized*—that is, the parser processes character and entity references (such as `é` or `<`) and replaces whitespace with spaces.

An `NMTOKEN` (or name token) is similar to `CDATA`, but most nonalphanumeric characters and internal whitespace are disallowed, and the parser removes leading and trailing whitespace. `NMTOKENS` is a whitespace-separated list of name tokens.

The `ID` construct is quite useful. An `ID` is a name token that must be unique in the document—the parser checks the uniqueness. You will see an application in the next sample program. An `IDREF` is a reference to an ID

that exists in the same document, which the parser also checks. `IDREFS` is a whitespace-separated list of ID references.

An `ENTITY` attribute value refers to an "unparsed external entity." That is a holdover from SGML that is rarely used in practice. The annotated XML specification at **https://www.xml.com/axml/axml.html** has an example.

A DTD can also define *entities*, or abbreviations that are replaced during parsing. You can find a good example for the use of entities in the user interface descriptions of the Firefox browser. Those descriptions are formatted in XML and contain entity definitions such as

```
<!ENTITY back.label "Back">
```

Elsewhere, text can contain an entity reference, for example:

```
<menuitem label="&back.label;"/>
```

The parser replaces the entity reference with the replacement string. To internationalize the application, only the string in the entity definition needs to be changed. Other uses of entities are more complex and less common; look at the XML specification for details.

This concludes the introduction to DTDs. Now that you have seen how to use DTDs, you can configure your parser to take advantage of them.

First, tell the document builder factory to turn on validation:

```
factory.setValidating(true);
```

All builders produced by this factory validate their input against a DTD. The most useful benefit of validation is ignoring whitespace in element content. For example, consider the XML fragment

```
<font>
   <name>Helvetica</name>
   <size>36</size>
</font>
```

A nonvalidating parser reports the whitespace between the `font`, `name`, and `size` elements because it has no way of knowing if the children of `font` are

```
(name,size)
(#PCDATA,name,size)*
```

or perhaps

```
ANY
```

Once the DTD specifies that the children are `(name,size)`, the parser knows that the whitespace between them is not text. Call

```
factory.setIgnoringElementContentWhitespace(true);
```

and the builder will stop reporting the whitespace in text nodes. That means you can now *rely on* the fact that a `font` node has two children. You no longer need to program a tedious loop:

```
for (int i = 0; i < children.getLength(); i++) {
    Node child = children.item(i);
    if (child instanceof Element childElement) {
        if (childElement.getTagName().equals("name")) . . .;
        else if (childElement.getTagName().equals("size")) . . .;
    }
}
```

Instead, you can simply access the first and second child:

```
var nameElement = (Element) children.item(0);
var sizeElement = (Element) children.item(1);
```

That is why DTDs are so useful. You don't overload your program with rule-checking code—the parser has already done that work by the time you get the document.

When the parser reports an error, your application will want to do something about it—log it, show it to the user, or throw an exception to abandon the parsing. Therefore, you should install an error handler whenever you use validation. Supply an object that implements the `ErrorHandler` interface. That interface has three methods:

```
void warning(SAXParseException e)
void error(SAXParseException e)
void fatalError(SAXParseException e)
```

Install the error handler with the `setErrorHandler` method of the `DocumentBuilder` class:

```
builder.setErrorHandler(handler);
```

`javax.xml.parsers.DocumentBuilder` **1.4**

- `void setEntityResolver(EntityResolver resolver)`
  sets the resolver to locate entities that are referenced in the XML documents being parsed.
- `void setErrorHandler(ErrorHandler handler)`
  sets the handler to report errors and warnings that occur during parsing.

org.xml.sax.EntityResolver **1.4**

- `public InputSource resolveEntity(String publicID, String systemID)`
  returns an input source that contains the data referenced by the given ID(s), or `null` to indicate that this resolver doesn't know how to resolve the particular name. The `publicID` parameter may be `null` if no public ID was supplied.

`org.xml.sax.InputSource` 1.4

- `InputSource(InputStream in)`
- `InputSource(Reader in)`
- `InputSource(String systemID)`

construct an input source from a stream, reader, or system ID (usually a relative or absolute URL).

org.xml.sax.ErrorHandler 1.4

- `void fatalError(SAXParseException e)`
- `void error(SAXParseException e)`
- `void warning(SAXParseException e)`

Override these methods to provide handlers for fatal errors, nonfatal errors, and warnings.

`org.xml.sax.SAXParseException` 1.4

- `int getLineNumber()`
- `int getColumnNumber()`

return the line and column numbers of the end of the processed input that caused the exception.

`javax.xml.catalog.CatalogManager` 9

- `static CatalogResolver catalogResolver(CatalogFeatures features, URI... uris)`

produces a resolver that uses the catalog files located at the provided URIs. This class implements `EntityResolver` as well as the resolver classes used by StAX, schema validation, and XSL transforms.

`javax.xml.catalog.CatalogFeatures` 9

- `static CatalogFeatures defaults()`

yields an instance with default settings.

`javax.xml.parsers.DocumentBuilderFactory` 1.4

- `boolean isValidating()`
- `void setValidating(boolean value)`

get or set the `validating` property of the factory. If set to `true`, the parsers that this factory generates validate their input.

- `boolean isIgnoringElementContentWhitespace()`
- `void setIgnoringElementContentWhitespace(boolean value)`

get or set the `ignoringElementContentWhitespace` property of the factory. If set to `true`, the parsers that this factory generates ignore whitespace between element nodes that don't have mixed content (i.e., a mixture of elements and `#PCDATA`).

3.4.2. XML Schema

XML Schema is quite a bit more complex than the DTD syntax, so we will only cover the basics. For more information, we recommend the tutorial at **https://www.w3.org/TR/xmlschema-0**.

To reference a schema file in a document, add attributes to the root element, for example:

```
<?xml version="1.0"?>
<config xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
      xsi:noNamespaceSchemaLocation="config.xsd">
   . . .
</config>
```

This declaration states that the schema file `config.xsd` should be used to validate the document. If your document uses namespaces, the syntax is a bit more complex—see the XML Schema tutorial for details. (The prefix `xsi` is a *namespace alias*; see **Section 3.6** for more information.)

A schema defines a *type* for each element and attribute. A *simple type* is a string, perhaps with restrictions on its contents. Everything else is a *complex* type. An element with a simple type can have no attributes and no child elements. Otherwise, it must have a complex type. Conversely, attributes always have a simple type.

Some simple types are built into XML Schema, including

```
xsd:string
xsd:int
xsd:boolean
```



Note: We use the prefix `xsd:` to denote the XML Schema Definition namespace. Some authors use the prefix `xs:` instead.

You can define your own simple types. For example, here is an enumerated type:

```
<xsd:simpleType name="StyleType">
   <xsd:restriction base="xsd:string">
      <xsd:enumeration value="PLAIN" />
      <xsd:enumeration value="BOLD" />
      <xsd:enumeration value="ITALIC" />
      <xsd:enumeration value="BOLD_ITALIC" />
   </xsd:restriction>
</xsd:simpleType>
```

When you define an element, you specify its type:

```
<xsd:element name="name" type="xsd:string"/>
<xsd:element name="size" type="xsd:int"/>
<xsd:element name="style" type="StyleType"/>
```

The type constrains the element content. For example, the elements

```
<size>10</size>
<style>PLAIN</style>
```

will validate correctly, but the elements

```
<size>default</size>
```

```
<style>SLANTED</style>
```

will be rejected by the parser.

You can compose types into complex types, for example:

```
<xsd:complexType name="FontType">
   <xsd:sequence>
      <xsd:element ref="name"/>
      <xsd:element ref="size"/>
      <xsd:element ref="style"/>
   </xsd:sequence>
</xsd:complexType>
```

A `FontType` is a sequence of `name`, `size`, and `style` elements. In this type definition, we use the `ref` attribute and refer to definitions that are located elsewhere in the schema. You can also nest definitions, like this:

```
<xsd:complexType name="FontType">
   <xsd:sequence>
      <xsd:element name="name" type="xsd:string"/>
      <xsd:element name="size" type="xsd:int"/>
      <xsd:element name="style">
         <xsd:simpleType>
            <xsd:restriction base="xsd:string">
               <xsd:enumeration value="PLAIN" />
               <xsd:enumeration value="BOLD" />
               <xsd:enumeration value="ITALIC" />
               <xsd:enumeration value="BOLD_ITALIC" />
            </xsd:restriction>
         </xsd:simpleType>
      </xsd:element>
   </xsd:sequence>
</xsd:complexType>
```

Note the *anonymous type definition* of the `style` element.

The `xsd:sequence` construct is the equivalent of the concatenation notation in DTDs. The `xsd:choice` construct is the equivalent of the `|` operator. For example,

```
<xsd:complexType name="contactinfo">
   <xsd:choice>
      <xsd:element ref="email"/>
      <xsd:element ref="phone"/>
   </xsd:choice>
</xsd:complexType>
```

This is the equivalent of the DTD type `email|phone`.

To allow repeated elements, use the `minoccurs` and `maxoccurs` attributes. For example, the equivalent of the DTD type `item*` is

```
<xsd:element name="item" type=". . ." minoccurs="0" maxoccurs="unbounded">
```

To specify attributes, add `xsd:attribute` elements to `complexType` definitions:

```
<xsd:element name="size">
   <xsd:complexType>
      . . .
      <xsd:attribute name="unit" type="xsd:string" use="optional" default="cm"/>
   </xsd:complexType>
</xsd:element>
```

This is the equivalent of the DTD statement

```
<!ATTLIST size unit CDATA #IMPLIED "cm">
```

Enclose element and type definitions of your schema inside an `xsd:schema` element:

```
<xsd:schema xmlns:xsd="http://www.w3.org/2001/XMLSchema">
   . . .
</xsd:schema>
```

Parsing an XML file with a schema is similar to parsing a file with a DTD, but with two differences:

1. You need to turn on support for namespaces, even if you don't use them in your XML files.

   ```
   factory.setNamespaceAware(true);
   ```

2. You need to prepare the factory for handling schemas, with the following magic incantation:

   ```
   final String JAXP_SCHEMA_LANGUAGE =
         "http://java.sun.com/xml/jaxp/properties/schemaLanguage";
   final String W3C_XML_SCHEMA = "http://www.w3.org/2001/XMLSchema";
   factory.setAttribute(JAXP_SCHEMA_LANGUAGE, W3C_XML_SCHEMA);
   ```

3.4.3. A Practical Example

In this section, we work through a practical example that shows the use of XML in a realistic setting.

Suppose an application needs configuration data that specifies arbitrary objects, not just text strings. We provide two mechanisms for instantiating the object: with a constructor, and with a factory method. Here is how to make a `Color` object using a constructor:

```
<construct class="java.awt.Color">
   <int>55</int>
   <int>200</int>
   <int>100</int>
</construct>
```

Here is an example with a factory method:

```
<factory class="java.nio.file.Path" method="of">
   <string>logging.properties</string>
</factory>
```

If the factory method name is omitted, it defaults to `getInstance`.

As you can see, there are elements for describing strings and integers. We also support the `boolean` type, and other primitive types can be added in the same way.

Just to show off the syntax, there is a second mechanism for primitive types:

```
<value type="int">30</value>
```

A configuration is a sequence of entries. Each entry has an ID and an object:

```
<config>
   <entry id="background">
      <construct class="java.awt.Color">
         <value type="int">55</value>
         <value type="int">200</value>
         <value type="int">100</value>
      </construct>
   </entry>
   . . .
</config>
```

The parser checks that IDs are unique.

The program in **Listing 3.2** shows how to parse a configuration file. A sample configuration is defined in **Listing 3.3**.

The program uses the schema instead of the DTD if you choose a file that contains the string `-schema`.

The DTD, shown in **Listing 3.4**, is straightforward.

Listing 3.5 contains the equivalent schema. In the schema, we can provide additional checking: an `int` or `boolean` element can only contain integer or boolean content. Note the use of the `xsd:group` construct to define parts of complex types that are used repeatedly.

This example is a typical use of XML. The XML format is robust enough to express complex relationships. The XML parser adds value by taking over the routine job of validity checking and supplying defaults.

Listing 3.2 v2ch03/read/XMLReadDemo.java

```
1  package v2ch03.read;
2
3  import module java.base;
4  import module java.xml;
5  import  org.w3c.dom.CharacterData;
```

```
6
7  /**
8   * This program shows how to use an XML file to describe Java objects.
9   */
10 class XMLReadDemo {
11     void main(String[] args) throws Exception {
12         String filename = args.length == 0 ? IO.readln("Input file: ") : args[0];
13
14         DocumentBuilderFactory factory = DocumentBuilderFactory.newInstance();
15         factory.setValidating(true);
16
17         if (filename.contains("-schema")) {
18             factory.setNamespaceAware(true);
19             final String JAXP_SCHEMA_LANGUAGE
20                 = "http://java.sun.com/xml/jaxp/properties/schemaLanguage";
21             final String W3C_XML_SCHEMA = "http://www.w3.org/2001/XMLSchema";
22             factory.setAttribute(JAXP_SCHEMA_LANGUAGE, W3C_XML_SCHEMA);
23         }
24
25         factory.setIgnoringElementContentWhitespace(true);
26
27         DocumentBuilder builder = factory.newDocumentBuilder();
28
29         builder.setErrorHandler(new ErrorHandler() {
30             public void warning(SAXParseException e) throws SAXException {
31                 System.err.println("Warning: " + e.getMessage());
32             }
33
34             public void error(SAXParseException e) throws SAXException {
35                 System.err.println("Error: " + e.getMessage());
36                 System.exit(0);
37             }
38
39             public void fatalError(SAXParseException e) throws SAXException {
40                 System.err.println("Fatal error: " + e.getMessage());
41                 System.exit(0);
42             }
43         });
44
45         Document doc = builder.parse(filename);
46         Map<String, Object> config = parseConfig(doc.getDocumentElement());
47         IO.println(config);
48     }
49
50     Map<String, Object> parseConfig(Element e) throws ReflectiveOperationException {
51         var result = new HashMap<String, Object>();
52         NodeList children = e.getChildNodes();
53         for (int i = 0; i < children.getLength(); i++) {
54             var child = (Element) children.item(i);
55             String name = child.getAttribute("id");
56             Object value = parseObject((Element) child.getFirstChild());
57             result.put(name, value);
58         }
59         return result;
60     }
61
62     Object parseObject(Element e) throws ReflectiveOperationException {
63         String tagName = e.getTagName();
```

```
        if (tagName.equals("factory"))
            return parseFactory(e);
        else if (tagName.equals("construct"))
            return parseConstruct(e);
        else {
            String childData = ((CharacterData) e.getFirstChild()).getData();
            if (tagName.equals("int"))
                return Integer.valueOf(childData);
            else if (tagName.equals("boolean"))
                return Boolean.valueOf(childData);
            else
                return childData;
        }
    }

    Object parseFactory(Element e) throws ReflectiveOperationException {
        String className = e.getAttribute("class");
        String methodName = e.getAttribute("method");
        Object[] args = parseArgs(e.getChildNodes());
        Class<?>[] parameterTypes = getParameterTypes(args);
        Method method = Class.forName(className).getMethod(methodName, parameterTypes);
        return method.invoke(null, args);
    }

    Object parseConstruct(Element e) throws ReflectiveOperationException {
        String className = e.getAttribute("class");
        Object[] args = parseArgs(e.getChildNodes());
        Class<?>[] parameterTypes = getParameterTypes(args);
        Constructor<?> constructor = Class.forName(className).getConstructor(parameterTypes);
        return constructor.newInstance(args);
    }

    Object[] parseArgs(NodeList elements) throws ReflectiveOperationException {
        var result = new Object[elements.getLength()];
        for (int i = 0; i < result.length; i++)
            result[i] = parseObject((Element) elements.item(i));
        return result;
    }

    Map<Class<?>, Class<?>> toPrimitive = Map.of(Integer.class, int.class, Boolean.class,
            boolean.class);

    Class<?>[] getParameterTypes(Object[] args) {
        var result = new Class<?>[args.length];
        for (int i = 0; i < result.length; i++) {
            Class<?> cl = args[i].getClass();
            result[i] = toPrimitive.get(cl);
            if (result[i] == null) result[i] = cl;
        }
        return result;
    }
}
```

Listing 3.3 v2ch03/read/config.xml

```
<?xml version="1.0"?>
<!DOCTYPE config SYSTEM "config.dtd">
<config>
  <entry id="background">
    <construct class="java.awt.Color">
      <int>55</int>
      <int>200</int>
      <int>100</int>
    </construct>
  </entry>
  <entry id="currency">
    <factory class="java.util.Currency">
      <string>USD</string>
    </factory>
  </entry>
</config>
```

Listing 3.4 v2ch03/read/config.dtd

```
<!ELEMENT config (entry)*>

<!ELEMENT entry (string|int|boolean|construct|factory)>
<!ATTLIST entry id ID #IMPLIED>

<!ELEMENT construct (string|int|boolean|construct|factory)*>
<!ATTLIST construct class CDATA #IMPLIED>

<!ELEMENT factory (string|int|boolean|construct|factory)*>
<!ATTLIST factory class CDATA #IMPLIED>
<!ATTLIST factory method CDATA "getInstance">

<!ELEMENT string (#PCDATA)>
<!ELEMENT int (#PCDATA)>
<!ELEMENT boolean (#PCDATA)>
```

Listing 3.5 v2ch03/read/config.xsd

```
<xsd:schema xmlns:xsd="http://www.w3.org/2001/XMLSchema">
   <xsd:element name="config">
      <xsd:complexType>
         <xsd:sequence>
            <xsd:element name="entry" minOccurs="0" maxOccurs="unbounded">
               <xsd:complexType>
                  <xsd:group ref="Object"/>
                  <xsd:attribute name="id" type="xsd:ID"/>
               </xsd:complexType>
            </xsd:element>
         </xsd:sequence>
      </xsd:complexType>
   </xsd:element>

   <xsd:element name="construct">
      <xsd:complexType>
```

```
         <xsd:group ref="Arguments"/>
         <xsd:attribute name="class" type="xsd:string"/>
      </xsd:complexType>
   </xsd:element>

   <xsd:element name="factory">
      <xsd:complexType>
         <xsd:group ref="Arguments"/>
         <xsd:attribute name="class" type="xsd:string"/>
         <xsd:attribute name="method" type="xsd:string" default="getInstance"/>
      </xsd:complexType>
   </xsd:element>

   <xsd:group name="Object">
      <xsd:choice>
         <xsd:element ref="construct"/>
         <xsd:element ref="factory"/>
         <xsd:element name="string" type="xsd:string"/>
         <xsd:element name="int" type="xsd:int"/>
         <xsd:element name="boolean" type="xsd:boolean"/>
      </xsd:choice>
   </xsd:group>

   <xsd:group name="Arguments">
      <xsd:sequence>
         <xsd:group ref="Object" minOccurs="0" maxOccurs="unbounded"/>
      </xsd:sequence>
   </xsd:group>
</xsd:schema>
```

3.5. Locating Information with XPath

If you want to locate a specific piece of information in an XML document, it can be a bit of a hassle to navigate the nodes of the DOM tree. The XPath language makes it simple to access tree nodes. For example, suppose you have this XHTML document:

```
<html>
   <head>
      . . .
      <title>. . .</title>
      . . .
   </head>
   . . .
</html>
```

You can get the title text by evaluating the XPath expression

```
/html/head/title/text()
```

That's a lot simpler than the plain DOM approach:

1. Get the document root.
2. Get the first child and cast it as an `Element`.

3. Locate the `title` element among its children.
4. Get its first child and cast it as a `CharacterData` node.
5. Get its data.

An XPath can describe *a set of nodes* in an XML document. For example, the XPath

```
/html/body/form
```

describes the set of all `form` elements that are children of the `body` element in an XHTML file. You can select a particular element with the `[]` operator:

```
/html/body/form[1]
```

is the first form. (The index values start at 1.)

Use the `@` operator to get attribute values. The XPath expression

```
/html/body/form[1]/@action
```

describes the `action` attribute of the first form. The XPath expression

```
/html/body/form/@action
```

describes all `action` attribute nodes of all `form` elements that are children of the `body` element.

There are a number of useful XPath functions. For example,

```
count(/html/body/form)
```

returns the number of `form` children of the `body` element. There are many more elaborate XPath expressions; see the specification at **https://www.w3.org/TR/xpath** or the online tutorial at **https://www.zvon.org/xxl/XPathTutorial/General/examples.html**.

To evaluate XPath expressions, first create an `XPath` object from an `XPathFactory`:

```
XPathFactory xpfactory = XPathFactory.newInstance();
path = xpfactory.newXPath();
```

Then, call the `evaluate` method to evaluate XPath expressions:

```
String username = path.evaluate("/html/head/title/text()", doc);
```

You can use the same `XPath` object to evaluate multiple expressions.

This form of the `evaluate` method returns a string result. It is suitable for retrieving text, such as the text child of the `title` element in the preceding example. If an XPath expression yields multiple nodes, make a call such

as the following:

```
XPathNodes result = path.evaluateExpression("/html/body/form", doc, XPathNodes.class);
```

The `XPathNodes` class is similar to a `NodeList`, but it extends the `Iterable` interface, allowing you to use an enhanced `for` loop.

If the result is a single node, use one of the following calls:

```
Node node = path.evaluateExpression("/html/body/form[1]", doc, Node.class);
node = (Node) path.evaluate("/html/body/form[1]", doc, XPathConstants.NODE);
```

If the result is a number, use:

```
int count = path.evaluateExpression("count(/html/body/form)", doc, Integer.class);
count = ((Number) path.evaluate("count(/html/body/form)",
        doc, XPathConstants.NUMBER)).intValue();
```

You don't have to start the search at the document root; you can start at any node or node list. For example, if you have a node from a previous evaluation, you can call

```
String result = path.evaluate(expression, node);
```

If you do not know the result of evaluating an XPath expression (perhaps because it comes from a user), then call

```
XPathEvaluationResult<?> result = path.evaluateExpression(expression, doc);
```

The expression `result.type()` is one of the constants

```
STRING
NODESET
NODE
NUMBER
BOOLEAN
```

of the `XPathEvaluationResult.XPathResultType` enumeration. Call `result.value()` to get the value.

The program in **Listing 3.6** demonstrates evaluation of arbitrary XPath expressions. Specify the URL of an XML page and type expressions such as `/html/body/h1[1]/text()`. The results of the expressions are displayed.

Listing 3.6 v2ch03/xpath/XPathDemo.java

```
1  package v2ch03.xpath;
2
3  import module java.base;
4  import module java.xml;
5
6  /**
```

```
 * This program evaluates XPath expressions.
 */
class XPathDemo {
    void main(String[] args) throws Exception {
        DocumentBuilderFactory factory = DocumentBuilderFactory.newInstance();
        DocumentBuilder builder = factory.newDocumentBuilder();

        // Avoid a delay in parsing an XHTML file--see the first note in Section 3.4.1
        builder.setEntityResolver(CatalogManager.catalogResolver(CatalogFeatures.defaults(),
                Path.of("v2ch03", "xpath", "catalog.xml").toAbsolutePath().toUri()));

        XPathFactory xpfactory = XPathFactory.newInstance();
        XPath path = xpfactory.newXPath();

        String urlString;
        if (args.length == 0) {
            urlString = "https://www.horstmann.com/index.html";
            IO.println("Using " + urlString);
        }
        else
            urlString = args[0];
        Document doc = builder.parse(urlString);
        boolean done = false;
        while (!done) {
            IO.print("XPath expression (empty line to exit): ");
            String expression = IO.readln();
            if (expression.strip().isEmpty()) {
                done = true;
            }
            else {
                try {
                    XPathEvaluationResult<?> result = path.evaluateExpression(expression, doc);
                    if (result.type() == XPathEvaluationResult.XPathResultType.NODESET) {
                        for (Node n : (XPathNodes) result.value())
                            IO.println(description(n));
                    }
                    else if (result.type() == XPathEvaluationResult.XPathResultType.NODE)
                        IO.println((Node) result.value());
                    else
                        IO.println(result.value());
                }
                catch (XPathExpressionException e) {
                    IO.println(e.getMessage());
                }
            }
        }
    }

    String description(Node n) {
        return switch (n) {
            case Element _ -> "Element " + n.getNodeName();
            case Attr _ -> "Attribute " + n;
            default -> n.toString();
        };
    }
}
```

`javax.xml.xpath.XPathFactory` 5.0

- `static XPathFactory newInstance()`
  returns an `XPathFactory` instance for creating `XPath` objects.
- `XPath newXpath()`
  constructs an `XPath` object for evaluating XPath expressions.

javax.xml.xpath.XPath 5.0

- `String evaluate(String expression, Object startingPoint)`
  evaluates an expression, beginning at the given starting point. The starting point can be a node or node list. If the result is a node or node set, the returned string consists of the data of all text node children.
- `Object evaluate(String expression, Object startingPoint, QName resultType)`
  evaluates an expression, beginning at the given starting point. The starting point can be a node or node list. The `resultType` is one of the constants `STRING`, `NODE`, `NODESET`, `NUMBER`, or `BOOLEAN` in the `XPathConstants` class. The return value is a `String`, `Node`, `NodeList`, `Number`, or `Boolean`.
- `<T> T evaluateExpression(String expression, Object item, Class<T> type)` 9
  evaluates the given expression and yields the result as a value of the given type.
- `XPathEvaluationResult<?> evaluateExpression(String expression, InputSource source)` 9
  evaluates the given expression.

javax.xml.xpath.XPathEvaluationResult<T> 9

- `XPathEvaluationResult.XPathResultType type()`
  returns one of the enumeration constants `STRING`, `NODESET`, `NODE`, `NUMBER`, `BOOLEAN`.
- `T value()`
  returns the result value.

3.6. Using Namespaces

The Java language uses packages to avoid name clashes. Programmers can use the same name for different classes as long as they aren't in the same package. XML has a similar *namespace* mechanism for element and attribute names.

A namespace is identified by a Uniform Resource Identifier (URI). Here are three examples:

```
http://www.w3.org/2001/XMLSchema
uuid:1c759aed-b748-475c-ab68-10679700c4f2
urn:com:books-r-us
```

The HTTP URL form is the most common. Note that the URL is just used as an identifier string, not as a locator for a document. For example, the namespace identifiers

```
http://www.horstmann.com/corejava
http://www.horstmann.com/corejava/index.html
```

denote *different* namespaces, even though a web server would serve the same document for both URLs.

There need not be any document at a namespace URL—the XML parser doesn't attempt to find anything at that location. However, as a help to programmers who encounter a possibly unfamiliar namespace, it is customary to place a document explaining the purpose of the namespace at the URL location. For example, if you point your browser to the namespace URL for the XML Schema namespace (`http://www.w3.org/2001/XMLSchema`), you will find a document describing the XML Schema standard.

Why use HTTP URLs for namespace identifiers? It is easy to ensure that they are unique. If you choose a real URL, the host part's uniqueness is guaranteed by the domain name system. Your organization can then arrange for the uniqueness of the remainder of the URL. This is the same rationale that underlies the use of reversed domain names in Java package names.

Of course, although long namespace identifiers are good for uniqueness, you don't want to deal with long identifiers any more than you have to. In the Java programming language, you use the `import` mechanism to specify the long names of packages, and then use just the short class names. In XML, there is a similar mechanism:

```
<tagName xmlns="namespaceURI">
    children
</tagName>
```

The element and its children are now part of the given namespace.

A child can provide its own namespace, for example:

```
<tagName xmlns="namespaceURI">
    <childTagName xmlns="childNamespaceURI">
        grandchildren
    </childTagName>
    more children
</tagName>
```

Then the first child and the grandchildren are part of the second namespace.

This simple mechanism works well if you need only a single namespace or if the namespaces are naturally nested. Otherwise, you will want to use a second mechanism that has no analog in Java. You can have a *prefix* for a namespace—a short identifier that you choose for a particular document. Here is a typical example—the `xsd` prefix in an XML Schema file:

```
<xsd:schema xmlns:xsd="http://www.w3.org/2001/XMLSchema">
   <xsd:element name="config"/>
   . . .
</xsd:schema>
```

The attribute

```
xmlns:prefix="namespaceURI"
```

defines a namespace and a prefix. In our example, the prefix is the string `xsd`. Thus, `xsd:schema` really means `schema` in the namespace `http://www.w3.org/2001/XMLSchema`.



Note: Only child elements inherit the namespace of their parent. Attributes without an explicit prefix are never part of a namespace. Consider this contrived example:

```
<configuration xmlns="http://www.horstmann.com/corejava"
   xmlns:si="http://www.bipm.fr/enus/3_SI/si.html">
   <size value="210" si:unit="mm"/>
   . . .
</configuration>
```

In this example, the elements `configuration` and `size` are part of the namespace with URI `http://www.horstmann.com/corejava`. The attribute `si:unit` is part of the namespace with URI `http://www.bipm.fr/enus/3_SI/si.html`. However, the attribute `value` is not part of any namespace.

You can control how the parser deals with namespaces. By default, the DOM parser of the Java XML API is not namespace-aware.

To turn on namespace handling, call the `setNamespaceAware` method of the `DocumentBuilderFactory`:

```
DocumentBuilderFactory factory = DocumentBuilderFactory.newInstance();
factory.setNamespaceAware(true);
```

Now, all builders the factory produces support namespaces.

Alternatively, since Java 13, call

```
DocumentBuilderFactory factory = DocumentBuilderFactory.newNSInstance();
```

Each node created by a builder has three properties:

- The *qualified name*, with a prefix, returned by `getNodeName`, `getTagName`, and so on
- The namespace URI, returned by the `getNamespaceURI` method
- The *local name*, without a prefix or a namespace, returned by the `getLocalName` method

Here is an example. Suppose the parser sees the following element:

```
<xsd:schema xmlns:xsd="http://www.w3.org/2001/XMLSchema">
```

It then reports the following:

- Qualified name = `xsd:schema`
- Namespace URI = `http://www.w3.org/2001/XMLSchema`
- Local name = `schema`



Note: If namespace awareness is turned off, `getNamespaceURI` and `getLocalName` return `null`.

org.w3c.dom.Node 1.4

- `String getLocalName()`
  returns the local name (without prefix), or `null` if the parser is not namespace-aware.
- `String getNamespaceURI()`
  returns the namespace URI, or `null` if the node is not part of a namespace or if the parser is not namespace-aware.



Caution: In the Java API, namespaces interact poorly with XPath expressions. You need to manually install a "namespace context" to resolve namespace prefixes. Moreover, Java still uses XPath 1.0, so you can't use the `*:tagName` syntax of XPath 2.0. The following context uses an `_` instead:

```
path.setNamespaceContext(new NamespaceContext() {
    public String getNamespaceURI(String prefix) {
        return doc.lookupNamespaceURI(prefix.equals("_") ? null : prefix);
    }
    public Iterator<String> getPrefixes(String val) { return null; }
    public String getPrefix(String uri) { return null; }
});
```

Use `_` for the default namespace and prefixes for the others, such as `//_:div/svg:svg`.

`javax.xml.parsers.DocumentBuilderFactory` 1.4

- `static DocumentBuilderFactory newNSInstance()` 13
  yields a factory whose builders are namespace-aware.
- `boolean isNamespaceAware()`
- `void setNamespaceAware(boolean value)`
  get or set the `namespaceAware` property of the factory. If set to `true`, the parsers that this factory generates are namespace-aware.

3.7. Streaming Parsers

The DOM parser reads an XML document in its entirety into a tree data structure. For most practical applications, DOM works fine. However, it can be inefficient if the document is large and if your processing algorithm is simple enough that you can analyze nodes on the fly, without having to see all of the tree structure. In these cases, you should use a streaming parser.

In the following sections, we discuss two streaming parsers supplied by the Java API. The SAX parser uses event callbacks, and the StAX parser provides an iterator through the parsing events. The latter is usually a bit more convenient.

3.7.1. Using the SAX Parser

The SAX parser reports events as it parses the components of the XML input, but it does not store the document in any way—it is up to the event handlers to build a data structure. In fact, the DOM parser is built on top of the SAX parser. It builds the DOM tree as it receives the parser events.

Whenever you use a SAX parser, you need a handler that defines the event actions for the various parse events. The `ContentHandler` interface defines several callback methods that the parser executes as it parses the docu-

ment. Here are the most important ones:

- `startElement` and `endElement` are called each time a start tag or end tag is encountered.
- `characters` is called whenever character data are encountered.
- `startDocument` and `endDocument` are called once each, at the start and the end of the document.

For example, when parsing the fragment

```
<font>
   <name>Helvetica</name>
   <size units="pt">36</size>
</font>
```

the parser makes the following callbacks:

1. `startElement`, element name: `font`
2. `startElement`, element name: `name`
3. `characters`, content: `Helvetica`
4. `endElement`, element name: `name`
5. `startElement`, element name: `size`, attributes: `units="pt"`
6. `characters`, content: `36`
7. `endElement`, element name: `size`
8. `endElement`, element name: `font`

Your handler needs to override these methods and have them carry out whatever action you want to carry out as you parse the file. The program at the end of this section prints all links of the form `<a href=". . .">` in an HTML file. It simply overrides the `startElement` method of the handler to check for elements with name `a` and an attribute with name `href`. This is potentially useful for implementing a "web crawler"—a program that reaches more and more web pages by following links.



Note: HTML doesn't have to be valid XML, and most web pages deviate so much from proper XML that the example programs will not be able to parse them. However, the pages on my web site `https://horstmann.com` are written in XHTML (an HTML dialect that is proper XML). You can use those pages to test the example program. For example, if you run

```
java v2ch03.sax.SAXDemo https://horstmann.com/corejava/index.html
```

you will see a list of the URLs of all links on that page.

The sample program is a good example for the use of SAX. We don't care at all in which context the `a` elements occur, and there is no need to store a tree structure.

Here is how you get a SAX parser:

```
SAXParserFactory factory = SAXParserFactory.newInstance();
SAXParser parser = factory.newSAXParser();
```

You can now process a document:

```
parser.parse(source, handler);
```

Here, `source` can be a file, URL string, or input stream. The `handler` belongs to a subclass of `DefaultHandler`. The `DefaultHandler` class defines do-nothing methods for the four interfaces:

```
ContentHandler
DTDHandler
EntityResolver
ErrorHandler
```

The example program defines a handler that overrides the `startElement` method of the `ContentHandler` interface to watch out for `a` elements with an `href` attribute:

```
var handler = new DefaultHandler() {
        public void startElement(String namespaceURI, String lname, String qname,
                Attributes attrs) throws SAXException {
            if (lname.equalsIgnoreCase("a") && attrs != null) {
                String href = attrs.getValue("href");
                if (href != null) IO.println(href);
            }
        }
    };
```

The `startElement` method has three parameters that describe the element name. The `qname` parameter reports the qualified name of the form `prefix:localname`. If namespace processing is turned on, then the `namespaceURI` and `lname` parameters provide the namespace and local name (without the namespace).

As with the DOM parser, namespace processing is turned off by default. To activate namespace processing, call the `setNamespaceAware` method of the factory class. Alternatively, as of Java 13, make a namespace-aware factory with `newNSInstance`:

```
SAXParserFactory factory = SAXParserFactory.newNSInstance();
SAXParser saxParser = factory.newSAXParser();
```

In this program, we cope with another common issue. An XHTML file starts with a tag that contains a DTD reference, and the parser will want to load it. Understandably, the W3C isn't too happy to serve billions of copies of files such as `www.w3.org/TR/xhtml1/DTD/xhtml1-strict.dtd`. At one point, they refused altogether, but at the time of this writing, they serve the DTD at a glacial pace. If you don't need to validate the document, just call

```
factory.setFeature(
   "http://apache.org/xml/features/nonvalidating/load-external-dtd", false);
```

Listing 3.7 contains the code for the web crawler program. Later in this chapter, you will see another interesting use of SAX. An easy way of turning a non-XML data source into XML is to report the SAX events that an XML parser would report. See **Section 3.9** for details.

Listing 3.7 v2ch03/sax/SAXDemo.java

```
package v2ch03.sax;

import module java.base;
import module java.xml;
import org.xml.sax.Attributes;

/**
 * This program demonstrates how to use a SAX parser. The program prints all hyperlinks of an
 * XHTML web page.
 */
class SAXDemo {
    void main(String[] args) throws Exception {
        String url;
        if (args.length == 0) {
            url = "https://horstmann.com/corejava/index.html";
            IO.println("Using " + url);
        }
        else
            url = args[0];

        var handler = new DefaultHandler() {
            public void startElement(String namespaceURI, String lname, String qname,
                    Attributes attrs) {
                if (lname.equals("a") && attrs != null) {
                    String href = attrs.getValue("href");
                    if (href != null) IO.println(href);
                }
            }
        };

        SAXParserFactory factory = SAXParserFactory.newInstance();
        factory.setNamespaceAware(true);
        factory.setFeature("http://apache.org/xml/features/nonvalidating/load-external-dtd",
                false);
        SAXParser saxParser = factory.newSAXParser();
        InputStream in = new URI(url).toURL().openStream();
        saxParser.parse(in, handler);
    }
}
```

`javax.xml.parsers.SAXParserFactory` 1.4

- `static SAXParserFactory newInstance()`
- `static SAXParserFactory newNSInstance()` 13

  return an instance of the `SAXParserFactory` class. The second method yields a namespace-aware instance.
- `SAXParser newSAXParser()`

  returns an instance of the `SAXParser` class.
- `boolean isNamespaceAware()`
- `void setNamespaceAware(boolean value)`

  get or set the `namespaceAware` property of the factory. If set to `true`, the parsers that this factory generates are namespace-aware.

- `boolean isValidating()`
- `void setValidating(boolean value)`
  get or set the `validating` property of the factory. If set to `true`, the parsers that this factory generates validate their input.

`javax.xml.parsers.SAXParser` **1.4**

- `void parse(File f, DefaultHandler handler)`
- `void parse(String url, DefaultHandler handler)`
- `void parse(InputStream in, DefaultHandler handler)`
  parse an XML document from the given file, URL, or input stream and report parse events to the given handler.

org.xml.sax.ContentHandler **1.4**

- `void startDocument()`
- `void endDocument()`
  are called at the start or the end of the document.
- `void startElement(String uri, String lname, String qname, Attributes attr)`
- `void endElement(String uri, String lname, String qname)`
  are called at the start or the end of an element. If the parser is namespace-aware, they report the URI of the namespace, the local name without prefix, and the qualified name with prefix.
- `void characters(char[] data, int start, int length)`
  is called when the parser reports character data.

org.xml.sax.Attributes **1.4**

- `int getLength()`
  returns the number of attributes stored in this attribute collection.
- `String getLocalName(int index)`
  returns the local name (without prefix) of the attribute with the given index, or the empty string if the parser is not namespace-aware.
- `String getURI(int index)`
  returns the namespace URI of the attribute with the given index, or the empty string if the node is not part of a namespace or if the parser is not namespace-aware.
- `String getQName(int index)`
  returns the qualified name (with prefix) of the attribute with the given index, or the empty string if the qualified name is not reported by the parser.
- `String getValue(int index)`
- `String getValue(String qname)`
- `String getValue(String uri, String lname)`
  return the attribute value from a given index, qualified name, or namespace URI + local name. Return `null` if the value doesn't exist.

3.7.2. Using the StAX Parser

The StAX parser is a "pull parser." Instead of installing an event handler, you simply iterate through the events, using this basic loop:

```
InputStream in = url.openStream();
XMLInputFactory factory = XMLInputFactory.newInstance();
XMLStreamReader parser = factory.createXMLStreamReader(in);
while (parser.hasNext()) {
    int event = parser.next();
    Call parser methods to obtain event details
}
```

For example, when parsing the fragment

```
<font>
   <name>Helvetica</name>
   <size units="pt">36</size>
</font>
```

the parser yields the following events:

1. `START_ELEMENT`, element name: `font`
2. `CHARACTERS`, content: white space
3. `START_ELEMENT`, element name: `name`
4. `CHARACTERS`, content: `Helvetica`
5. `END_ELEMENT`, element name: `name`
6. `CHARACTERS`, content: white space
7. `START_ELEMENT`, element name: `size`
8. `CHARACTERS`, content: `36`
9. `END_ELEMENT`, element name: `size`
10. `CHARACTERS`, content: white space
11. `END_ELEMENT`, element name: `font`

To analyze the attribute values, call the appropriate methods of the `XMLStreamReader` class. For example,

```
String units = parser.getAttributeValue(null, "units");
```

gets the `units` attribute of the current element.

By default, namespace processing is enabled. You can deactivate it by modifying the factory:

```
XMLInputFactory factory = XMLInputFactory.newInstance();
factory.setProperty(XMLInputFactory.IS_NAMESPACE_AWARE, false);
```

Listing 3.8 contains the code for the web crawler program implemented with the StAX parser. As you can see, the code is simpler than the equivalent SAX code because you don't have to worry about event handling.

Listing 3.8 v2ch03/stax/StAXDemo.java

```
1  package v2ch03.stax;
2
3  import module java.base;
4  import module java.xml;
```

```
5
6  /**
7   * This program demonstrates how to use a StAX parser. The program prints all hyperlinks of an
8   * XHTML web page.
9   */
10 class StAXDemo {
11     void main(String[] args) throws Exception {
12         String urlString;
13         if (args.length == 0) {
14             urlString = "https://www.horstmann.com/index.html";
15             IO.println("Using " + urlString);
16         }
17         else
18             urlString = args[0];
19         InputStream in = new URI(urlString).toURL().openStream();
20         XMLInputFactory factory = XMLInputFactory.newInstance();
21         XMLStreamReader parser = factory.createXMLStreamReader(in);
22         while (parser.hasNext()) {
23             int event = parser.next();
24             if (event == XMLStreamConstants.START_ELEMENT) {
25                 if (parser.getLocalName().equals("a")) {
26                     String href = parser.getAttributeValue(null, "href");
27                     if (href != null) IO.println(href);
28                 }
29             }
30         }
31     }
32 }
```

`javax.xml.stream.XMLInputFactory` 6

- `static XMLInputFactory newInstance()`
  returns an instance of the `XMLInputFactory` class.
- `void setProperty(String name, Object value)`
  sets a property for this factory, or throws an `IllegalArgumentException` if the property is not supported or cannot be set to the given value. The JDK implementation supports the following Boolean-valued properties:

`"javax.xml.stream.isValidating"`	When `false` (the default), the document is not validated. Not required by the specification.
`"javax.xml.stream.isNamespaceAware"`	When `true` (the default), namespaces are processed. Not required by the specification.
`"javax.xml.stream.isCoalescing"`	When `false` (the default), adjacent character data are not coalesced.
`"javax.xml.stream.isReplacingEntityReferences"`	When `true` (the default), entity references are replaced and reported as character data.

`"javax.xml.stream.isSupportingExternalEntities"`	When `true` (the default), external entities are resolved. The specification gives no default for this property.
`"javax.xml.stream.supportDTD"`	When `true` (the default), DTDs are reported as events.

- `XMLStreamReader createXMLStreamReader(InputStream in)`
- `XMLStreamReader createXMLStreamReader(InputStream in, String characterEncoding)`
- `XMLStreamReader createXMLStreamReader(Reader in)`
- `XMLStreamReader createXMLStreamReader(Source in)`

create a parser that reads from the given stream, reader, or JAXP source.

javax.xml.stream.XMLStreamReader 6

- `boolean hasNext()`
  returns `true` if there is another parse event.
- `int next()`
  sets the parser state to the next parse event and returns one of the following constants: `START_ELEMENT`, `END_ELEMENT`, `CHARACTERS`, `START_DOCUMENT`, `END_DOCUMENT`, `CDATA`, `COMMENT`, `SPACE` (ignorable whitespace), `PROCESSING_INSTRUCTION`, `ENTITY_REFERENCE`, `DTD`.
- `boolean isStartElement()`
- `boolean isEndElement()`
- `boolean isCharacters()`
- `boolean isWhiteSpace()`
  return `true` if the current event is a start element, end element, character data, or whitespace.
- `QName getName()`
- `String getLocalName()`
  get the name of the element in a `START_ELEMENT` or `END_ELEMENT` event.
- `String getText()`
  returns the characters of a `CHARACTERS`, `COMMENT`, or `CDATA` event, the replacement value for an `ENTITY_REFERENCE`, or the internal subset of a `DTD`.
- `int getAttributeCount()`
- `QName getAttributeName(int index)`
- `String getAttributeLocalName(int index)`
- `String getAttributeValue(int index)`
  get the attribute count and the names and values of the attributes, provided the current event is `START_ELEMENT`.
- `String getAttributeValue(String namespaceURI, String name)`
  gets the value of the attribute with the given name, provided the current event is `START_ELEMENT`. If `namespaceURI` is `null`, the namespace is not checked.

3.8. Generating XML Documents

You now know how to write Java programs that read XML. Let us now turn to the opposite process: producing XML output. Of course, you could write an XML file simply by making a sequence of `print` calls, printing the elements, attributes, and text content, but that would not be a good idea. The code is rather tedious, and you can easily make mistakes if you don't pay attention to special symbols (such as `"` or `<`) in the attribute values and text content.

A better approach is to build up a DOM tree with the contents of the document and then write out the tree contents. The following sections discuss the details.

3.8.1. Documents without Namespaces

To build a DOM tree, you start out with an empty document. You can get an empty document by calling the `newDocument` method of the `DocumentBuilder` class:

```
Document doc = builder.newDocument();
```

Use the `createElement` method of the `Document` class to construct the elements of your document:

```
Element rootElement = doc.createElement(rootName);
Element childElement = doc.createElement(childName);
```

Use the `createTextNode` method to construct text nodes:

```
Text textNode = doc.createTextNode(textContents);
```

Add the root element to the document, and add the child nodes to their parents:

```
doc.appendChild(rootElement);
rootElement.appendChild(childElement);
childElement.appendChild(textNode);
```

As you build up the DOM tree, you may also need to set element attributes. Simply call the `setAttribute` method of the `Element` class:

```
rootElement.setAttribute(name, value);
```

3.8.2. Documents with Namespaces

If you use namespaces, the procedure for creating a document is slightly different.

First, set the builder factory to be namespace-aware, then create the builder:

```
DocumentBuilderFactory factory = DocumentBuilderFactory.newInstance();
factory.setNamespaceAware(true);
builder = factory.newDocumentBuilder();
```

Then use `createElementNS` instead of `createElement` to create any nodes:

```
String namespace = "http://www.w3.org/2000/svg";
Element rootElement = doc.createElementNS(namespace, "svg");
```

If your node has a qualified name with a namespace prefix, then any necessary `xmlns`-prefixed attributes are created automatically. For example, if you need SVG inside XHTML, you can construct an element like this:

```
Element svgElement = doc.createElement(namespace, "svg:svg")
```

When the element is written, it turns into

```
<svg:svg xmlns:svg="http://www.w3.org/2000/svg">
```

If you need to set element attributes whose names are in a namespace, use the `setAttributeNS` method of the `Element` class:

```
rootElement.setAttributeNS(namespace, qualifiedName, value);
```

3.8.3. Writing Documents

Somewhat curiously, it is not so easy to write a DOM tree to an output stream. The easiest approach is to use the Extensible Stylesheet Language Transformations (XSLT) API. For more information about XSLT, turn to **Section 3.9**. Right now, consider the code that follows a magic incantation to produce XML output.

We apply the do-nothing transformation to the document and capture its output. To include a `DOCTYPE` node in the output, we also need to set the `SYSTEM` and `PUBLIC` identifiers as output properties.

```
// construct the do-nothing transformation
Transformer t = TransformerFactory.newInstance().newTransformer();
// set output properties to get a DOCTYPE node
t.setOutputProperty(OutputKeys.DOCTYPE_SYSTEM, systemIdentifier);
t.setOutputProperty(OutputKeys.DOCTYPE_PUBLIC, publicIdentifier);
// set indentation
t.setOutputProperty(OutputKeys.INDENT, "yes");
t.setOutputProperty(OutputKeys.METHOD, "xml");
t.setOutputProperty("{http://xml.apache.org/xslt}indent-amount", "2");
// apply the do-nothing transformation and send the output to a file
t.transform(new DOMSource(doc), new StreamResult(new FileOutputStream(file)));
```

Another approach is to use the `LSSerializer` interface. To get an instance, you have to use the following magic incantation:

```
DOMImplementation impl = doc.getImplementation();
var implLS = (DOMImplementationLS) impl.getFeature("LS", "3.0");
LSSerializer ser = implLS.createLSSerializer();
```

If you want spaces and line breaks, set this flag:

```
ser.getDomConfig().setParameter("format-pretty-print", true);
```

Then it's simple enough to convert a document to a string:

```
String str = ser.writeToString(doc);
```

If you want to write the output directly to a file, you need an `LSOutput`:

```
LSOutput out = implLS.createLSOutput();
out.setEncoding("UTF-8");
out.setByteStream(Files.newOutputStream(path));
ser.write(doc, out);
```

`javax.xml.parsers.DocumentBuilder` 1.4

- `Document newDocument()`
  returns an empty document.

org.w3c.dom.Document 1.4

- `Element createElement(String name)`
- `Element createElementNS(String uri, String qname)`
  create an element with the given name.
- `Text createTextNode(String data)`
  creates a text node with the given data.

org.w3c.dom.Node 1.4

- `Node appendChild(Node child)`
  appends a node to the list of children of this node. Returns the appended node.

org.w3c.dom.Element 1.4

- `void setAttribute(String name, String value)`
- `void setAttributeNS(String uri, String qname, String value)`
  set the attribute with the given name to the given value. If the qualified name has an alias prefix, then `uri` must not be `null`.

`javax.xml.transform.TransformerFactory` 1.4

- `static TransformerFactory newInstance()`
  returns an instance of the `TransformerFactory` class.
- `Transformer newTransformer()`
  returns an instance of the `Transformer` class that carries out an identity (do-nothing) transformation.

`javax.xml.transform.Transformer` 1.4

- `void setOutputProperty(String name, String value)`
  sets an output property. See **https://www.w3.org/TR/xslt#output** for a listing of the standard output properties. The most useful ones are shown here:

  `doctype-public` The public ID to be used in the `DOCTYPE` declaration

`doctype-system`	The system ID to be used in the `DOCTYPE` declaration
`indent`	`"yes"` or `"no"`
`method`	`"xml"`, `"html"`, `"text"`, or a custom string

- `void transform(Source from, Result to)`
  transforms an XML document.

`javax.xml.transform.dom.DOMSource` 1.4

- `DOMSource(Node n)`
  constructs a source from the given node. Usually, `n` is a document node.

`javax.xml.transform.stream.StreamResult` 1.4

- `StreamResult(File f)`
- `StreamResult(OutputStream out)`
- `StreamResult(Writer out)`
- `StreamResult(String systemID)`
  construct a stream result from a file, stream, writer, or system ID (usually a relative or absolute URL).

3.8.4. Writing an XML Document with StAX

In the preceding section, you saw how to produce an XML document by writing a DOM tree. If you have no other use for the DOM tree, that approach is not very efficient.

The StAX API lets you write an XML tree directly. Construct an `XMLStreamWriter` from an `OutputStream`:

```
XMLOutputFactory factory = XMLOutputFactory.newInstance();
XMLStreamWriter writer = factory.createXMLStreamWriter(out);
```

To produce the XML header, call

```
writer.writeStartDocument();
```

Then call

```
writer.writeStartElement(name);
```

Add attributes by calling

```
writer.writeAttribute(name, value);
```

Now you can add child elements by calling `writeStartElement` again, or write characters with

```
writer.writeCharacters(text);
```

When you have written all child nodes, call

```
writer.writeEndElement();
```

This causes the current element to be closed.

To write an element without children (such as `<img . . ./>`), use the call

```
writer.writeEmptyElement(name);
```

Finally, at the end of the document, call

```
writer.writeEndDocument();
```

This call closes any open elements.

You still need to close the `XMLStreamWriter`, and you need to do it manually since the `XMLStreamWriter` interface does not extend the `AutoCloseable` interface.

As with the DOM/XSLT approach, you don't have to worry about escaping characters in attribute values and character data. However, it is possible to produce malformed XML, such as a document with multiple root nodes. Also, the current version of StAX has no support for producing indented output.

Listing 3.9 demonstrates two ways of producing XML: by constructing and saving a DOM tree, and by directly writing the XML with the StAX API.

The program draws a modernist painting—a random set of colored rectangles (see **Figure 3.3**). To save our masterpiece, we use the Scalable Vector Graphics (SVG) format. SVG is an XML format to describe complex graphics in a device-independent fashion. You can find more information about SVG at **https://www.w3.org/Graphics/SVG**. To view SVG files, simply use any modern browser.

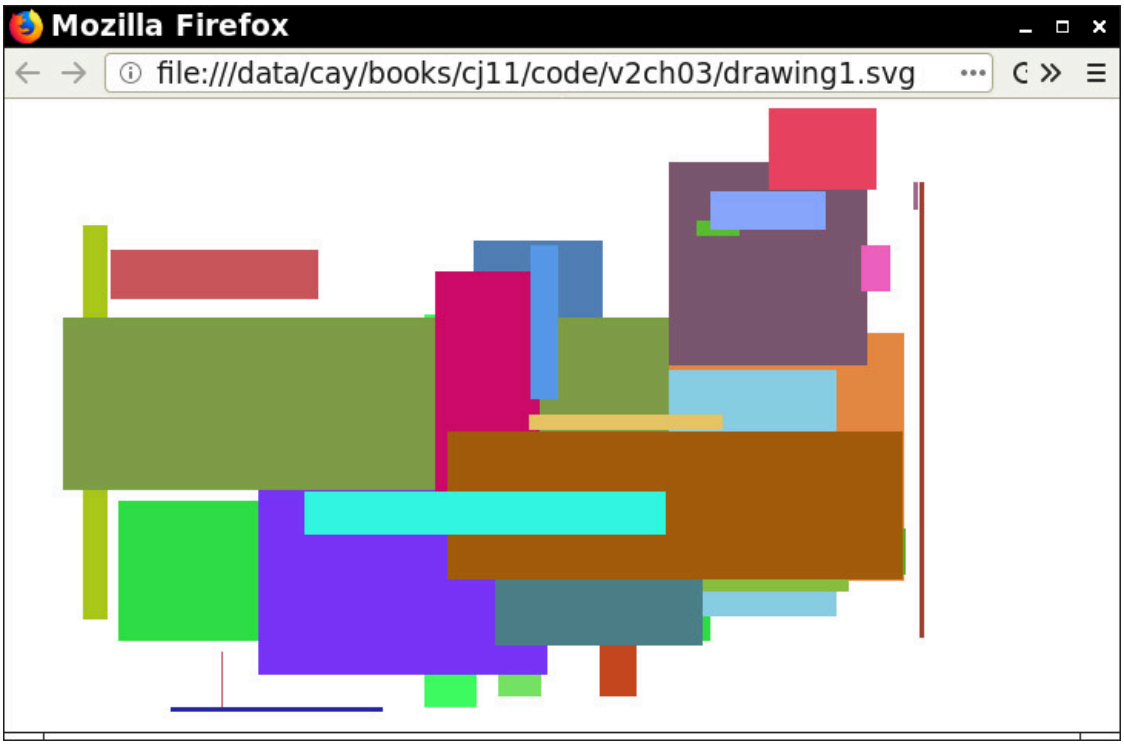


Figure 3.3: Generating modern art

We don't need to go into details about SVG; for our purposes, we just need to know how to express a set of colored rectangles. Here is a sample:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE svg PUBLIC "-//W3C//DTD SVG 20000802//EN"
   "http://www.w3.org/TR/2000/CR-SVG-20000802/DTD/svg-20000802.dtd">
<svg xmlns="http://www.w3.org/2000/svg" width="300" height="150">
   <rect x="231" y="61" width="9" height="12" fill="#6e4a13"/>
   <rect x="107" y="106" width="56" height="5" fill="#c406be"/>
   . . .
</svg>
```

As you can see, each rectangle is described as a `rect` node. The position, width, height, and fill color are attributes. The fill color is an RGB value in hexadecimal.



Note: SVG uses attributes heavily. In fact, some attributes are quite complex. For example, here is a path element:

```
<path d="M 100 100 L 300 100 L 200 300 z">
```

The `M` denotes a "moveto" command, `L` is "lineto," and `z` is "closepath" (!). Apparently, the designers of this data format didn't have much confidence in using XML for structured data. In your own XML formats, you might want to use elements instead of complex attributes. Or, if you find that XML is too verbose for your purpose, choose a different format.

Listing 3.9 v2ch03/write/XMLWriteDemo.java

```
1  package v2ch03.write;
2
3  import module java.base;
4  import module java.xml;
```

```
5
6  /**
7   * This program shows how to write an XML file. It produces modern art in SVG format.
8   */
9  class XMLWriteDemo {
10     void main(String[] args) throws Exception {
11         Document doc = newDrawing(600, 400);
12         writeDocument(doc, "drawing1.svg");
13         writeNewDrawing(600, 400, "drawing2.svg");
14     }
15
16     RandomGenerator generator = RandomGenerator.getDefault();
17
18     /**
19      * Creates a new random drawing.
20      * @param drawingWidth the width of the drawing in pixels
21      * @param drawingHeight the height of the drawing in pixels
22      * @return the DOM tree of the SVG document
23      */
24     Document newDrawing(int drawingWidth, int drawingHeight)
25             throws ParserConfigurationException {
26         DocumentBuilderFactory factory = DocumentBuilderFactory.newInstance();
27         factory.setNamespaceAware(true);
28         DocumentBuilder builder = factory.newDocumentBuilder();
29         ;
30         String namespace = "http://www.w3.org/2000/svg";
31         Document doc = builder.newDocument();
32         Element svgElement = doc.createElementNS(namespace, "svg");
33         doc.appendChild(svgElement);
34         svgElement.setAttribute("width", "" + drawingWidth);
35         svgElement.setAttribute("height", "" + drawingHeight);
36         int n = 10 + generator.nextInt(20);
37         for (int i = 1; i <= n; i++) {
38             int x = generator.nextInt(drawingWidth);
39             int y = generator.nextInt(drawingHeight);
40             int width = generator.nextInt(drawingWidth - x);
41             int height = generator.nextInt(drawingHeight - y);
42             int r = generator.nextInt(256);
43             int g = generator.nextInt(256);
44             int b = generator.nextInt(256);
45
46             Element rectElement = doc.createElementNS(namespace, "rect");
47             rectElement.setAttribute("x", "" + x);
48             rectElement.setAttribute("y", "" + y);
49             rectElement.setAttribute("width", "" + width);
50             rectElement.setAttribute("height", "" + height);
51             rectElement.setAttribute("fill", "#%02x%02x%02x".formatted(r, g, b));
52             svgElement.appendChild(rectElement);
53         }
54         return doc;
55     }
56
57     /**
58      * Saves a document using DOM/XSLT.
59      * @param doc the document to be written
60      * @param filename the name of the destination file
61      */
62     void writeDocument(Document doc, String filename)
```

```
            throws TransformerException, IOException {
        Transformer t = TransformerFactory.newInstance().newTransformer();
        t.setOutputProperty(OutputKeys.DOCTYPE_SYSTEM,
                "http://www.w3.org/TR/2000/CR-SVG-20000802/DTD/svg-20000802.dtd");
        t.setOutputProperty(OutputKeys.DOCTYPE_PUBLIC, "-//W3C//DTD SVG 20000802//EN");
        t.setOutputProperty(OutputKeys.INDENT, "yes");
        t.setOutputProperty(OutputKeys.METHOD, "xml");
        t.setOutputProperty("{http://xml.apache.org/xslt}indent-amount", "2");
        t.transform(new DOMSource(doc),
                new StreamResult(Files.newOutputStream(Path.of(filename))));
    }

    /**
     * Uses StAX to write an SVG document with a random drawing.
     * @param drawingWidth the width of the drawing in pixels
     * @param drawingHeight the width of the drawing in pixels
     * @param filename the name of the destination file
     */
    void writeNewDrawing(int drawingWidth, int drawingHeight, String filename)
            throws XMLStreamException, IOException {
        XMLOutputFactory factory = XMLOutputFactory.newInstance();
        XMLStreamWriter writer = factory
                .createXMLStreamWriter(Files.newOutputStream(Path.of(filename)));
        writer.writeStartDocument();
        writer.writeDTD("""
                <!DOCTYPE svg PUBLIC "-//W3C//DTD SVG 20000802//EN"
                  "http://www.w3.org/TR/2000/CR-SVG-20000802/DTD/svg-20000802.dtd">
                """);
        writer.writeStartElement("svg");
        writer.writeDefaultNamespace("http://www.w3.org/2000/svg");
        writer.writeAttribute("width", "" + drawingWidth);
        writer.writeAttribute("height", "" + drawingHeight);
        int n = 10 + generator.nextInt(20);
        for (int i = 1; i <= n; i++) {
            int x = generator.nextInt(drawingWidth);
            int y = generator.nextInt(drawingHeight);
            int width = generator.nextInt(drawingWidth - x);
            int height = generator.nextInt(drawingHeight - y);
            int r = generator.nextInt(256);
            int g = generator.nextInt(256);
            int b = generator.nextInt(256);
            writer.writeEmptyElement("rect");
            writer.writeAttribute("x", "" + x);
            writer.writeAttribute("y", "" + y);
            writer.writeAttribute("width", "" + width);
            writer.writeAttribute("height", "" + height);
            writer.writeAttribute("fill", "#%02x%02x%02x".formatted(r, g, b));
        }
        writer.writeEndDocument(); // closes svg element
    }
}
```

`javax.xml.stream.XMLOutputFactory` 6

- `static XMLOutputFactory newInstance()`
  returns an instance of the `XMLOutputFactory` class.

- `XMLStreamWriter createXMLStreamWriter(OutputStream stream)`
- `XMLStreamWriter createXMLStreamWriter(OutputStream stream, String encoding)`
- `XMLStreamWriter createXMLStreamWriter(Writer writer)`
- `XMLStreamWriter createXMLStreamWriter(Result result)`

  create a writer that writes to the given stream, writer, or JAXP result.

javax.xml.stream.XMLStreamWriter 6

- `void writeStartDocument()`
- `void writeStartDocument(String xmlVersion)`
- `void writeStartDocument(String encoding, String xmlVersion)`

  write the XML processing instruction at the top of the document. Note that the `encoding` parameter is only used to write the attribute. It does not set the character encoding of the output.
- `void setDefaultNamespace(String namespaceURI)`
- `void setPrefix(String prefix, String namespaceURI)`

  set the default namespace or the namespace associated with a prefix. The declaration is scoped to the current element or, if no element has been written, to the document root.
- `void writeStartElement(String localName)`
- `void writeStartElement(String namespaceURI, String localName)`

  write a start tag, replacing the `namespaceURI` with the associated prefix.
- `void writeEndElement()`

  closes the current element.
- `void writeEndDocument()`

  closes all open elements.
- `void writeEmptyElement(String localName)`
- `void writeEmptyElement(String namespaceURI, String localName)`

  write a self-closing tag, replacing the `namespaceURI` with the associated prefix.
- `void writeAttribute(String localName, String value)`
- `void writeAttribute(String namespaceURI, String localName, String value)`

  write an attribute for the current element, replacing the `namespaceURI` with the associated prefix.
- `void writeCharacters(String text)`

  writes character data.
- `void writeCData(String text)`

  writes a `CDATA` block.
- `void writeDTD(String dtd)`

  writes the `dtd` string, which is assumed to contain a `DOCTYPE` declaration.
- `void writeComment(String comment)`

  writes a comment.
- `void close()`

  closes this writer.

3.9. XSL Transformations

The XSL Transformations (XSLT) mechanism allows you to specify rules for transforming XML documents into other formats, such as plain text, XHTML, or any other XML format. XSLT is commonly used to translate from one machine-readable XML format to another, or to translate XML into a presentation format for human consumption.

You need to provide an XSLT stylesheet that describes the conversion of XML documents into some other format. An XSLT processor reads an XML document and the stylesheet and produces the desired output (see **Figure 3.4**).

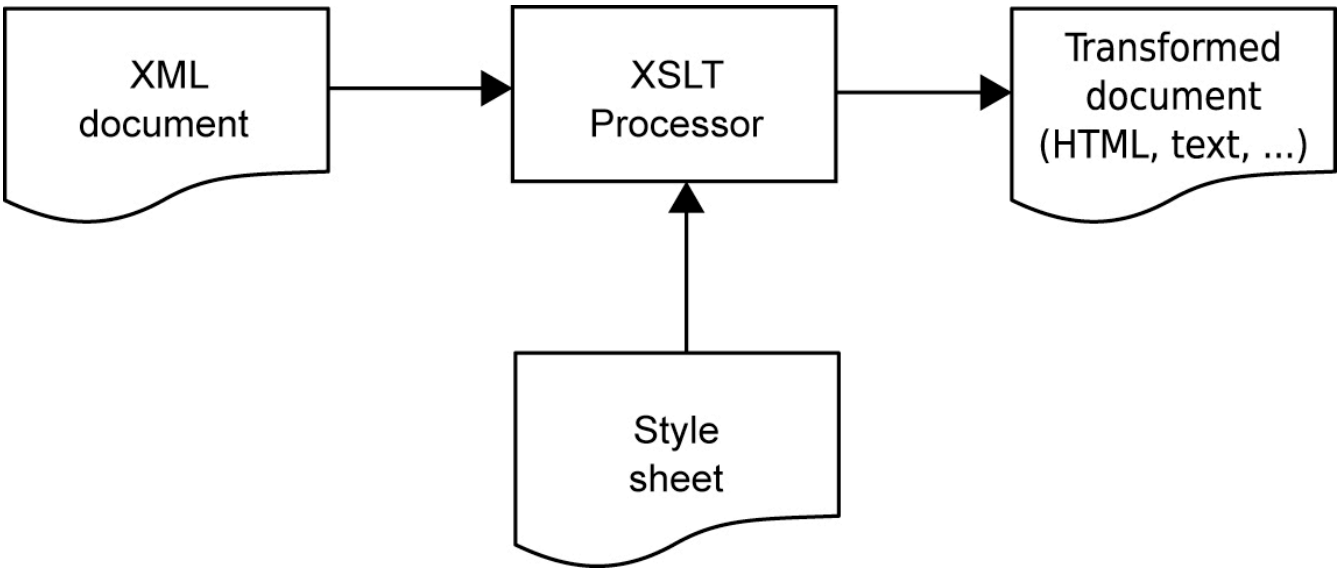


Figure 3.4: Applying XSL transformations

The XSLT specification is quite complex, and entire books have been written on the subject. We can't possibly discuss all the features of XSLT, so we will just work through a representative example. You can find more information in the book *Essential XML* by Don Box et al. The XSLT specification is available at **https://www.w3.org/TR/xslt**.

Suppose we want to transform XML files with employee records into HTML documents. Consider this input file:

```
<staff>
   <employee>
      <name>Carl Cracker</name>
      <salary>75000.0</salary>
      <hiredate year="1987" month="12" day="15"/>
   </employee>
   <employee>
      <name>Harry Hacker</name>
      <salary>50000.0</salary>
      <hiredate year="1989" month="10" day="1"/>
   </employee>
   <employee>
      <name>Tony Tester</name>
      <salary>40000.0</salary>
      <hiredate year="1990" month="3" day="15"/>
   </employee>
</staff>
```

The desired output is an HTML table:

```
<table border="1">
<tr>
<td>Carl Cracker</td><td>$75000.0</td><td>1987-12-15</td>
</tr>
<tr>
<td>Harry Hacker</td><td>$50000.0</td><td>1989-10-1</td>
</tr>
<tr>
```

```
<td>Tony Tester</td><td>$40000.0</td><td>1990-3-15</td>
</tr>
</table>
```

A stylesheet with transformation templates has this form:

```
<?xml version="1.0" encoding="ISO-8859-1"?>
<xsl:stylesheet
      xmlns:xsl="http://www.w3.org/1999/XSL/Transform"
      version="1.0">
   <xsl:output method="html"/>
   template1

   template2
   . . .
</xsl:stylesheet>
```

In our example, the `xsl:output` element specifies the method as HTML. Other valid method settings are `xml` and `text`.

Here is a typical template:

```
<xsl:template match="/staff/employee">
   <tr><xsl:apply-templates/></tr>
</xsl:template>
```

The value of the `match` attribute is an XPath expression. The template states: Whenever you see a node in the XPath set `/staff/employee`, do the following:

1. Emit the string `<tr>.`
2. Keep applying templates as you process its children.
3. Emit the string `</tr>` after you are done with all children.

In other words, this template generates the HTML table row markers around every employee record.

The XSLT processor starts processing by examining the root element. Whenever a node matches one of the templates, it applies the template. (If multiple templates match, the best matching one is used; see the specification at **https://www.w3.org/TR/xslt** for the gory details.) If no template matches, the processor carries out a default action. For text nodes, the default is to include the contents in the output. For elements, the default action is to create no output but to keep processing the children.

Here is a template for transforming `name` nodes in an employee file:

```
<xsl:template match="/staff/employee/name">
   <td><xsl:apply-templates/></td>
</xsl:template>
```

As you can see, the template produces the `<td>. . .</td>` delimiters, and it asks the processor to recursively visit the children of the `name` element. There is just one child—the text node. When the processor visits that node, it emits the text contents (provided, of course, that there is no other matching template).

You have to work a little harder if you want to copy attribute values into the output. Here is an example:

```
<xsl:template match="/staff/employee/hiredate">
   <td><xsl:value-of select="@year"/>-<xsl:value-of
   select="@month"/>-<xsl:value-of select="@day"/></td>
</xsl:template>
```

When processing a `hiredate` node, this template emits

1. The string `<td>`
2. The value of the `year` attribute
3. A hyphen
4. The value of the `month` attribute
5. A hyphen
6. The value of the `day` attribute
7. The string `</td>`

The `xsl:value-of` statement computes the string value of a node set. The node set is specified by the XPath value of the `select` attribute. In this case, the path is relative to the currently processed node. The node set is converted to a string by concatenation of the string values of all nodes. The string value of an attribute node is its value. The string value of a text node is its contents. The string value of an element node is the concatenation of the string values of its child nodes (but not its attributes).

Listing 3.10 contains the stylesheet for turning an XML file with employee records into an HTML table.

Listing 3.11 shows a different set of transformations. The input is the same XML file, and the output is plain text in the familiar property file format:

```
employee.1.name=Carl Cracker
employee.1.salary=75000.0
employee.1.hiredate=1987-12-15
employee.2.name=Harry Hacker
employee.2.salary=50000.0
employee.2.hiredate=1989-10-1
employee.3.name=Tony Tester
employee.3.salary=40000.0
employee.3.hiredate=1990-3-15
```

That example uses the `position()` function which yields the position of the current node as seen from its parent. We thus get an entirely different output simply by switching the stylesheet. This means you can safely use XML to describe your data; if some applications need the data in another format, just use XSLT to generate the alternative format.

It is simple to generate XSL transformations on the Java platform. Set up a transformer factory for each stylesheet. Then, get a transformer object and tell it to transform a source to a result:

```
var styleSheet = new File(filename);
var styleSource = new StreamSource(styleSheet);
Transformer t = TransformerFactory.newInstance().newTransformer(styleSource);
t.transform(source, result);
```

The arguments of the `transform` method are objects of classes that implement the `Source` and `Result` interfaces. Several classes implement the `Source` interface:

```
DOMSource
SAXSource
StAXSource
StreamSource
```

You can construct a `StreamSource` from a file, stream, reader, or URL, and a `DOMSource` from the node of a DOM tree. For example, in the preceding section, we invoked the identity transformation as

```
t.transform(new DOMSource(doc), result);
```

In our example program, we do something slightly more interesting. Instead of starting out with an existing XML file, we produce a SAX XML reader that gives the illusion of parsing an XML file by emitting appropriate SAX events. Actually, our XML reader reads a flat file, as described in **Chapter 1**. The input file looks like this:

```
Carl Cracker|75000.0|1987|12|15
Harry Hacker|50000.0|1989|10|1
Tony Tester|40000.0|1990|3|15
```

Our XML reader generates SAX events as it processes the input. Here is a part of the `parse` method of the `EmployeeReader` class that implements the `XMLReader` interface:

```
var attributes = new AttributesImpl();
handler.startDocument();
handler.startElement("", "staff", "staff", attributes);
boolean done = false;
while (!done) {
    String line = in.readLine();
    if (line == null) done = true;
    else {
        handler.startElement("", "employee", "employee", attributes);
        var tokenizer = new StringTokenizer(line, "|");
        handler.startElement("", "name", "name", attributes);
        String s = tokenizer.nextToken();
        handler.characters(s.toCharArray(), 0, s.length());
        handler.endElement("", "name", "name");
        . . .
        handler.endElement("", "employee", "employee");
    }
}
handler.endElement("", rootElement, rootElement);
handler.endDocument();
```

The `SAXSource` for the transformer is constructed from the XML reader:

```
t.transform(new SAXSource(new EmployeeReader(),
    new InputSource(new FileInputStream(filename))), result);
```

This is an ingenious trick to convert non-XML legacy data into XML. Of course, most XSLT applications will already have XML input data, and you can simply invoke the `transform` method on a `StreamSource`:

```
t.transform(new StreamSource(file), result);
```

The transformation result is an object of a class that implements the `Result` interface. The Java API supplies three classes:

```
DOMResult
SAXResult
StreamResult
```

To store the result in a DOM tree, use a `DocumentBuilder` to generate a new document node and wrap it into a `DOMResult`:

```
Document doc = builder.newDocument();
t.transform(source, new DOMResult(doc));
```

To save the output in a file, use a `StreamResult`:

```
t.transform(source, new StreamResult(file));
```

Listing 3.12 contains the complete source code.

Listing 3.10 v2ch03/transform/makehtml.xsl

```
1  <?xml version="1.0" encoding="ISO-8859-1"?>
2
3  <xsl:stylesheet
4     xmlns:xsl="http://www.w3.org/1999/XSL/Transform"
5     version="1.0">
6
7     <xsl:output method="html"/>
8
9     <xsl:template match="/staff">
10       <table border="1"><xsl:apply-templates/></table>
11    </xsl:template>
12
13    <xsl:template match="/staff/employee">
14       <tr><xsl:apply-templates/></tr>
15    </xsl:template>
16
17    <xsl:template match="/staff/employee/name">
18       <td><xsl:apply-templates/></td>
19    </xsl:template>
20
21    <xsl:template match="/staff/employee/salary">
22       <td>$<xsl:apply-templates/></td>
23    </xsl:template>
24
25    <xsl:template match="/staff/employee/hiredate">
```

```
      <td><xsl:value-of select="@year"/>-<xsl:value-of
      select="@month"/>-<xsl:value-of select="@day"/></td>
   </xsl:template>

</xsl:stylesheet>
```

Listing 3.11 v2ch03/transform/makeprop.xsl

```
<?xml version="1.0"?>

<xsl:stylesheet
   xmlns:xsl="http://www.w3.org/1999/XSL/Transform"
   version="1.0">

   <xsl:output method="text" omit-xml-declaration="yes"/>

   <xsl:template match="/staff/employee">
employee.<xsl:value-of select="position()"
/>.name=<xsl:value-of select="name/text()"/>
employee.<xsl:value-of select="position()"
/>.salary=<xsl:value-of select="salary/text()"/>
employee.<xsl:value-of select="position()"
/>.hiredate=<xsl:value-of select="hiredate/@year"
/>-<xsl:value-of select="hiredate/@month"
/>-<xsl:value-of select="hiredate/@day"/>
   </xsl:template>

</xsl:stylesheet>
```

Listing 3.12 v2ch03/transform/TransformDemo.java

```
package v2ch03.transform;

import module java.base;
import module java.xml;

import org.xml.sax.ContentHandler;
import v2ch07.util.Choices;

/**
 * This program demonstrates XSL transformations. It applies a transformation to a set of
 * employee records. The records are stored in the file employee.dat and turned into XML
 * format.
 */
class TransformDemo {
    void main(String[] args) throws Exception {
        Path path;
        String choice = Choices.choose("HTML", "Properties").toLowerCase();
        if (choice.equals("html"))
            path = Path.of("v2ch03", "transform", "makehtml.xsl");
        else
            path = Path.of("v2ch03", "transform", "makeprop.xsl");
        try (InputStream styleIn = Files.newInputStream(path)) {
            var styleSource = new StreamSource(styleIn);

            Transformer t = TransformerFactory.newInstance().newTransformer(styleSource);
```

```
            t.setOutputProperty(OutputKeys.INDENT, "yes");
            t.setOutputProperty(OutputKeys.METHOD, "xml");
            t.setOutputProperty("{http://xml.apache.org/xslt}indent-amount", "2");

            try (InputStream docIn = Files
                    .newInputStream(Path.of("v2ch03", "transform", "employee.dat"))) {
                t.transform(new SAXSource(new EmployeeReader(), new InputSource(docIn)),
                        new StreamResult(System.out));
            }
        }
    }
}

/**
 * This class reads the flat file employee.dat and reports SAX parser events to act as if it
 * was parsing an XML file.
 */
class EmployeeReader implements XMLReader {
    private ContentHandler handler;

    public void parse(InputSource source) throws IOException, SAXException {
        InputStream stream = source.getByteStream();
        var in = new BufferedReader(new InputStreamReader(stream));
        var atts = new AttributesImpl();

        if (handler == null) throw new SAXException("No content handler");

        handler.startDocument();
        handler.startElement("", "staff", "staff", atts);
        boolean done = false;
        while (!done) {
            String line = in.readLine();
            if (line == null)
                done = true;
            else {
                handler.startElement("", "employee", "employee", atts);
                var t = new StringTokenizer(line, "|");

                handler.startElement("", "name", "name", atts);
                String s = t.nextToken();
                handler.characters(s.toCharArray(), 0, s.length());
                handler.endElement("", "name", "name");

                handler.startElement("", "salary", "salary", atts);
                s = t.nextToken();
                handler.characters(s.toCharArray(), 0, s.length());
                handler.endElement("", "salary", "salary");

                atts.addAttribute("", "year", "year", "CDATA", t.nextToken());
                atts.addAttribute("", "month", "month", "CDATA", t.nextToken());
                atts.addAttribute("", "day", "day", "CDATA", t.nextToken());
                handler.startElement("", "hiredate", "hiredate", atts);
                handler.endElement("", "hiredate", "hiredate");
                atts.clear();

                handler.endElement("", "employee", "employee");
            }
        }
```

```
84
85          handler.endElement("", "staff", "staff");
86          handler.endDocument();
87      }
88
89      public void setContentHandler(ContentHandler newValue) {
90          handler = newValue;
91      }
92
93      public ContentHandler getContentHandler() {
94          return handler;
95      }
96
97      // the following methods are just do-nothing implementations
98      public void parse(String systemId) throws IOException, SAXException {
99      }
100
101     public void setErrorHandler(ErrorHandler handler) {
102     }
103
104     public ErrorHandler getErrorHandler() {
105         return null;
106     }
107
108     public void setDTDHandler(DTDHandler handler) {
109     }
110
111     public DTDHandler getDTDHandler() {
112         return null;
113     }
114
115     public void setEntityResolver(EntityResolver resolver) {
116     }
117
118     public EntityResolver getEntityResolver() {
119         return null;
120     }
121
122     public void setProperty(String name, Object value) {
123     }
124
125     public Object getProperty(String name) {
126         return null;
127     }
128
129     public void setFeature(String name, boolean value) {
130     }
131
132     public boolean getFeature(String name) {
133         return false;
134     }
135 }
```

`javax.xml.transform.TransformerFactory` **1.4**

- `Transformer newTransformer(Source styleSheet)`

returns an instance of the `Transformer` class that reads a stylesheet from the given source.

`javax.xml.transform.stream.StreamSource` 1.4

- `StreamSource(File f)`
- `StreamSource(InputStream in)`
- `StreamSource(Reader in)`
- `StreamSource(String systemID)`
  construct a stream source from a file, stream, reader, or system ID (usually a relative or absolute URL).

`javax.xml.transform.sax.SAXSource` 1.4

- `SAXSource(XMLReader reader, InputSource source)`
  constructs a SAX source that obtains data from the given input source and uses the given reader to parse the input.

org.xml.sax.XMLReader 1.4

- `void setContentHandler(ContentHandler handler)`
  sets the handler that is notified of parse events as the input is parsed.
- `void parse(InputSource source)`
  parses the input from the given input source and sends parse events to the content handler.

`javax.xml.transform.dom.DOMResult` 1.4

- `DOMResult(Node n)`
  constructs a source from the given node. Usually, `n` is a new document node.

`org.xml.sax.helpers.AttributesImpl` 1.4

- `void addAttribute(String uri, String lname, String qname, String type, String value)`
  adds an attribute to this attribute collection. The `lname` parameter is the local name without prefix, and `qname` is the qualified name with prefix. The `type` parameter is one of `"CDATA"`, `"ID"`, `"IDREF"`, `"IDREFS"`, `"NMTOKEN"`, `"NMTOKENS"`, `"ENTITY"`, `"ENTITIES"`, or `"NOTATION"`.
- `void clear()`
  removes all attributes from this attribute collection.

This example concludes our discussion of XML support in the Java API. You should now have a good perspective on the major strengths of XML—in particular, its automated parsing and validation as well as its powerful transformation mechanism. Of course, all this technology is only going to work for you if you design your XML formats well. You need to make sure that the formats are rich enough to express all your business needs, that they are stable over time, and that your business partners are willing to accept your XML documents. Those issues can be far more challenging than dealing with parsers, DTDs, or transformations.

In the next chapter, we will discuss network programming on the Java platform, starting with the basics of network sockets and moving on to higher-level protocols for e-mail and the World Wide Web.

Previous chapter
< 2. Input and Output

Next chapter
4. Networking >